MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

HE THONG THUONG MAI DIEN TU VOI TIM KIEM ANH CONG NGHE

Student: Nguyen Thanh Phat
Student ID: B220001
Class: DI2296A1
Advisor: TS. Tran Thi B

Can Tho, July 2026

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

HE THONG THUONG MAI DIEN TU VOI TIM KIEM ANH CONG NGHE

Student: Nguyen Thanh Phat
Student ID: B220001
Class: DI2296A1
Advisor: TS. Tran Thi B

Can Tho, 01/2026

EVALUATION OF ADVISOR

Advisor:

TS. Tran Thi B

ACKNOWLEDGEMENTS

I wish to express my deep gratitude to my advisor for their guidance throughout this thesis. I would also like to thank the lecturers at Can Tho University for their invaluable knowledge.

I am grateful to my family for their love and support, and to my friends for their encouragement.

Sincerely,

Can Tho, [Date]

[Your Name]

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CTU	Can Tho University
ICT	Information and Communication Technology
UI/UX	User Interface/User Experience
HTTP	Hypertext Transfer Protocol
CBIR	Content-Based Image Retrieval
CQRS	Command Query Responsibility Segregation
MediatR	Mediator library for .NET
EF Core	Entity Framework Core
JWT	JSON Web Token
pgvector	PostgreSQL vector extension

ABSTRACT

Write your abstract here (200-350 words as per CTU guidelines).

Structure:

1. Introduction to research topic and objectives
2. Main methodology and approach
3. Summary of results and findings
4. Main conclusions and recommendations

Keywords: e-commerce, content-based image retrieval, modular monolith, fashion retrieval, embedding models

PART 1: INTRODUCTION

1.1 CONTEXT AND PROBLEM STATEMENT

This thesis is submitted in partial fulfillment of the requirements for the degree of [**Bachelor of Engineering in Information Technology**]. It presents the complete analysis, design, implementation, and evaluation of ReSys.Shop — a fashion e-commerce platform with Content-Based Image Retrieval (CBIR) capabilities, featuring a comparative evaluation of multiple pretrained visual feature extraction models.

Fashion e-commerce represents one of the most competitive and technically demanding domains in online retail. Consumers expect rich visual experiences, personalized recommendations, and seamless checkout flows across multiple devices. Traditional text-based search often fails in fashion because shoppers struggle to articulate visual preferences (e.g., "a dress like this but in blue").

ReSys.Shop addresses three distinct problems simultaneously:

1. [**The user-facing problem**]: How can a fashion e-commerce platform provide intuitive visual search using modern machine learning techniques?
2. [**The engineering problem**]: How can a complex e-commerce system be architected to maintain modularity, testability, and operational clarity as it scales across 8 business domains?
3. [**The ML evaluation problem**]: Which pretrained visual feature extraction model (Fashion-CLIP, ResNet-50, EfficientNet-B0, CLIP-generic) offers the optimal balance of retrieval effectiveness and operational performance for fashion CBIR?

1.2 PROBLEM STATEMENT

Existing fashion e-commerce platforms typically fall into one of two categories:

1. [**Monolithic platforms**] (e.g., early Shopify, Magento) that become unmaintainable as business logic interleaves across features
2. [**Microservice platforms**] that introduce excessive operational overhead for small-to-medium teams, with distributed-transaction complexity for e-commerce workflows

Neither approach optimally serves a research context where rapid iteration on ML-powered features must coexist with stable transactional domains, the system

must be demonstrable as a single deployable unit, and code quality must be examinable and justifiable.

Table 1.

Table 1: Specific technical gaps identified

Gap	Evidence from prior art	Consequence
Exception-driven error handling	Typical ASP.NET controllers throw exceptions for validation failures	Unpredictable control flow
Anemic domain models	EF entities are data bags with no behavior	Business rules scattered across services
Horizontal layering	Controllers → Services → Repositories → Entities	Changes touch 4+ files
Tight module coupling	Services directly reference other modules' repositories	Cannot test modules in isolation
Missing vector search	Standard SQL databases cannot perform similarity search	Fashion image search requires separate infrastructure
No model comparison for CBIR	Prior art selects embedding models arbitrarily	Suboptimal model may be deployed

1.3 RELATED WORK

1.3.1 Content-Based Image Retrieval

Content-Based Image Retrieval (CBIR) systems retrieve images from a database based on visual similarity rather than textual metadata. Early CBIR systems relied on hand-crafted features such as color histograms, texture descriptors (Gabor filters), and shape representations (SIFT, SURF). These approaches suffered from the semantic gap — low-level visual features do not map reliably to high-level human concepts.

Deep learning transformed CBIR by learning visual representations directly from data. Convolutional Neural Networks (CNNs) such as ResNet and EfficientNet extract rich feature vectors that capture semantic content. More recently, Vision Transformers (ViT) and contrastive learning approaches like CLIP have enabled zero-shot visual retrieval by learning joint visual-linguistic representations.

1.3.2 Fashion Image Retrieval

Fashion-specific retrieval presents unique challenges: fine-grained visual differences (sleeve length, pattern, neckline), style consistency across views, and the need to match across diverse product categories. Fashion-CLIP extends CLIP with fashion-domain pretraining, achieving superior retrieval on fashion datasets compared to generic models.

Prior work in fashion CBIR typically selects a single embedding model without empirical comparison. This thesis addresses that gap by evaluating four models (Fashion-CLIP, ResNet-50, EfficientNet-B0, CLIP-generic) on both retrieval effectiveness and operational performance.

1.3.3 Modular Monolith Architecture

The modular monolith pattern combines the simplicity of a single deployable unit with the modularity benefits of microservices. Each business domain lives in an isolated module with explicit boundaries, communicating via in-process message dispatch rather than network calls.

Vertical slice architecture further refines this by organizing code around feature actions rather than technical layers. Each feature (e.g., "Create Product") is cohesively implemented in a single folder containing handler, endpoint, request, response, and validator — eliminating the cross-cutting concerns of traditional horizontal layering.

1.3.4 Vector Database Integration

PostgreSQL with the pgvector extension enables similarity search directly within the relational database, eliminating the need for a separate vector database. This approach leverages existing SQL tooling, transactions, and operational knowledge while providing cosine similarity and nearest-neighbor search on embedding vectors.

1.3.5 Summary

This thesis contributes a dual evaluation: (a) architectural patterns for modular e-commerce systems, and (b) empirical comparison of embedding models for fashion CBIR. The work bridges software engineering rigor with machine learning evaluation methodology.

1.4 OBJECTIVES

1.4.1 Primary Objectives

1. **Design and implement a modular monolith** with 8 self-contained business modules (Catalog, Identity, Inventory, Location, Ordering, Payment, Profile, Shipping) that communicate exclusively via in-process message dispatch, enforcing zero direct cross-module references.
2. **Apply a vertical-slice architecture** where each feature action (e.g., "Create Product", "Checkout Cart") is cohesively implemented in a single folder containing its handler, endpoint, request, response, and validator.
3. **Integrate Content-Based Image Retrieval (CBIR)** via a dedicated Python sidecar supporting multiple pretrained embedding models (Fashion-CLIP, ResNet-50, EfficientNet-B0, CLIP-generic), storing embeddings in PostgreSQL pgvector with a pluggable model interface.
4. **Enforce explicit error handling** through a `Result<T>` / `Error` type system that eliminates exceptions for control flow, making all failure paths explicit and traceable.
5. **Conduct comparative ML evaluation** to measure retrieval effectiveness (Precision@K, Recall@K, mAP) and operational performance (embedding generation time, query latency, storage footprint) across 4 embedding models on a fashion ground-truth dataset.

1.4.2 Secondary Objectives

1. Provide dual-channel frontends (Admin SPA and Storefront SPA) with distinct UI libraries optimized for their respective user roles.
2. Implement multi-provider abstractions for storage (Local/S3), notifications (SendGrid/SMTP/Sinch), payment gateways (Stripe/Bogus), and **embedding models** to demonstrate the Strategy pattern across both infrastructure and ML domains.
3. Achieve >70% unit-test coverage for domain logic and integration tests for all critical paths (checkout, payment webhooks, auth, CBIR search).

1.5 SCOPE AND DELIMITATIONS

1.5.1 Problem Boundary

This thesis addresses the **dual contribution** of (a) architectural design and implementation of a fashion e-commerce platform, and (b) comparative evaluation of pretrained visual embedding models for CBIR. The boundary is drawn around **software engineering process evidence** and **ML model evaluation methodology** — analysis, design, implementation, and comparative evaluation — rather than operational deployment or business operations.

1.5.2 In-Scope Deliverables

Table 2.

Table 2: In-scope deliverables and corresponding thesis chapters

Deliverable	Description	Thesis Chapter
Backend system	8 business modules (Catalog, Identity, Inventory, Location, Ordering, Payment, Profile, Shipping) implemented as vertical slices with CQRS, MediatR, and explicit `Result<T>` error handling	Chapters 3–7
Database design	PostgreSQL 17 + pgvector schema with per-module namespaces, vector embeddings for CBIR, EF Core migrations	Chapter 5
ML sidecar	Python FastAPI service with **pluggable embedding model interface** supporting Fashion-CLIP, ResNet-50, EfficientNet-B0, and CLIP-generic; HTTP API consumed by Catalog module	Chapters 3, 5, 7
ML model comparison	Comparative evaluation of 4 embedding models on retrieval effectiveness (Precision@K, Recall@K, mAP) and operational performance (latency, storage, memory)	Chapter 11
Dual-channel frontends	Vue 3 Admin SPA (PrimeVue) for administrators; Vue 3 Storefront SPA (Nuxt UI) for customers	Chapters 3, 6
Security stack	JWT bearer auth with rotation/reuse detection, permission-based authorization, rate limiting, anti-forgery, file upload guards	Chapter 8
Testing strategy	Unit tests (InMemory EF + Moq), integration tests (Testcontainers + WebApplicationFactory), frontend unit tests (Vitest), Python tests (pytest)	Chapter 10
Observability	OpenTelemetry traces/metrics/logs, correlation IDs, health checks, Hangfire background jobs	Chapters 3, 9
Design documentation	This thesis document set: 11 chapters of analysis, design rationale, and evaluation	All chapters

1.5.3 Out-of-Scope (Explicitly Deferred)

Table 3.

Table 3: Features explicitly deferred from thesis scope

Feature	Rationale	Impact on Thesis	Evidence
YARP API Gateway	SPA→API direct calls are sufficient for thesis demonstration; gateway adds ops complexity without research value	No architectural gap; API handles auth/rate limiting directly	`infra/Aspire/src/ReSys.AppHost/AppHost.cs:5-7`
Azure Blob Storage Provider	S3 + Local providers cover all thesis file-storage needs	Strategy pattern is demonstrated with 2 providers; adding a 3rd is mechanical	`appsettings.json:163-168` vs `Storage.Extensions.cs:79-82`
Facebook / Microsoft OAuth	Google OAuth demonstrates external-login architecture; adding more providers is identical pattern	No design gap; config blocks are disabled (`Enabled=false`)	`appsettings.json:48-57`
CI/CD Pipeline	Thesis evaluation is manual build/test; no production environment exists	Build/test commands are documented; CI is a deployment concern outside scope	`README.md:177-179`
Dockerfiles / Container Images	Aspire manages containers for local development only	Production containerization is a deployment extension, not a design contribution	`README.md:177`
Recommendation Engine (Collaborative Filtering)	CBIR with model comparison is the primary ML contribution; collaborative filtering is orthogonal	Listed as Future Work in Chapter 11	`Chapter 11 — Evaluation`
Payment Provider Beyond Stripe + Bogus	Two providers (real + dev stand-in) demonstrate the Strategy pattern adequately	No architectural gap	`Module/Payment/Services/Provider/`
Multi-tenancy / Multi-store	Current schema has `StoreId` columns but single-store logic	Database is forward-compatible; business logic extension is Future Work	`Order.cs:55`
Mobile Native Apps	Responsive web SPAs are the only client surfaces	Mobile is a separate client implementation using the same API	`app/Admin/`, `app/Store/`
Custom model training / fine-tuning	Using pretrained models only; training requires GPU cluster and dataset curation beyond thesis scope	Evaluation focuses on model **selection**, not model **creation**	`service/Embedding/pyproject.toml`

1.5.4 Scope Justification

The out-of-scope items share three characteristics:

1. **They do not affect the dual thesis contribution** — the architectural patterns (modular monolith, vertical slices, Result<T>) and the ML model comparison (Precision@K, Recall@K, mAP across 4 models) are fully demonstrable without them.
2. **They are additive, not structural** — each can be added later without redesigning existing modules (Strategy pattern, config blocks, provider patterns, additional embedding models).
3. **They shift focus from design/evaluation to operations** — CI/CD, Docker, gateway configuration, and model training are operational concerns rather than software architecture or ML evaluation contributions.

This scope aligns with the **principle of sufficient completeness for evaluation** (Shaw & Garlan, *Software Architecture*): the system must be complete enough to demonstrate its architectural properties and the ML model comparison, but need not be production-ready in every operational dimension.

1.6 STAKEHOLDERS

Table 4.

Table 4: Stakeholders and how their interests are addressed

Stakeholder	Interest	How addressed
Examiner / Thesis Committee	Evidence of structured SE process, design rationale, testability	This documentation set; clean architecture; comprehensive test suite
End Customer	Visual search, recommendations, smooth checkout	Storefront SPA; FashionCLIP CBIR; MediatR pipeline for reliable checkout
Administrator / Merchant	Product management, order fulfillment, user administration	Admin SPA; permission-based authorization; full CRUD on all modules
Future Developer / Researcher	Understandable, extensible codebase	Vertical slices; module isolation; `Result<T>` makes failure paths explicit; extensive inline docs

1.7 EVIDENCE

- `README.md:1-184` — project intent and WIP notes
- `AGENTS.md:1-80` — non-negotiable architectural rules
- `service/Api/src/Api/Program.cs:26-66` — composition root showing 8 modules
- `service/Api/src/Module/Catalog/Features/Admin/Products/Create/CreateProduct.cs:36-78` — vertical slice anatomy
- `service/Api/src/Shared/Application/Models/Results/Result.cs:1-43` — `Result<T>` type
- `Directory.Build.targets:42-53` — `ValidateVerticalSliceIsolation` (currently disabled, but intent is clear)
- `service/Embedding/src/main.py:1-29` — Python sidecar entry

1.8 THESIS OUTLINE

This thesis is organized into three parts:

Part 1: Introduction provides the problem context, related work in content-based image retrieval and modular architecture, research objectives, and methodology.

Part 2: Thesis Content presents the system design and implementation across ten chapters: requirements analysis, system architecture, domain design, database design, API design, detailed design, security design, deployment design, testing strategy, and implementation highlights.

Part 3: Conclusion evaluates the system against research objectives, traces requirements to implementation, summarizes contributions, and identifies future work directions.

PART 2: THESIS CONTENT

2 CHAPTER 1: REQUIREMENTS ANALYSIS

2.1 FUNCTIONAL REQUIREMENTS

Functional requirements are organized by business module. Each requirement is traceable to a vertical-slice feature folder or domain invariant in the codebase.

2.1.1 Catalog Module

Table 5.

Table 5: Catalog Module Functional Requirements

ID	Requirement	Priority	Evidence
CAT-FR-01	Administrators can create products with name, description, slug, SEO metadata, and fashion-specific fields (style code, season, material composition, care instructions, fit notes, department, gender target)	High	`Module/Catalog/Domain/Products/Product.cs:17-43`
CAT-FR-02	Products support variants (size/color combinations) with SKUs, barcodes, physical dimensions, and independent pricing	High	`Module/Catalog/Domain/Products/Variants/Variant.cs:14-69`
CAT-FR-03	Products can be classified via taxonomy (hierarchical categories: taxonomies → taxons)	High	`Module/Catalog/Features/Admin/Taxonomies/`, `Module/Catalog/Features/Admin/Taxons/`
CAT-FR-04	Products support option types (e.g., "Size", "Color") with option values (e.g., "Small", "Red")	High	`Module/Catalog/Domain/Products/Options/OptionType.cs`, `OptionValue.cs`
CAT-FR-05	Variant images can be uploaded, stored (Local/S3), and associated with variants	High	`Module/Catalog/Features/Admin/Products/Variants/Images/Upload/`
CAT-FR-06	**Image embeddings are generated via a pluggable ML sidecar** supporting Fashion-CLIP, ResNet-50, EfficientNet-B0, and CLIP-generic; embeddings stored in PostgreSQL pgvector with model metadata	High	`Module/Catalog/Features/Admin/Products/Variants/Images/Embeddings/`, `Shared/Operational/Persistence/Configurations/Vectors/Vector.Configuration.cs`
CAT-FR-07	Storefront users can search products by image (upload an image, find visually similar products)	High	`ApiTests/Catalog/Storefront/search-by-image.http`
CAT-FR-10	Embedding model is **configurable per deployment** via `EMBEDDING_MODEL` env var (e.g., `fashion-clip`, `resnet50`, `efficientnet_b0`, `clip`)	High	`service/Embedding/src/models/clip_model.py` (refactored to strategy pattern)
CAT-FR-08	Product status lifecycle: Draft → Active → Archived, with availability and discontinuation dates	Medium	`Module/Catalog/Domain/Products/Product.cs:20,31-33`
CAT-FR-09	Slug uniqueness enforced across the catalog	High	`Module/Catalog/Features/Admin/Products/Create/CreateProduct.cs:41-43`

2.1.2 Identity Module

Table 6.

Table 6: Identity Module Functional Requirements

ID	Requirement	Priority	Evidence
ID-FR-01	Users can register with email and password	High	`Module/Identity/Features/Store/Auth/Register/`
ID-FR-02	Users can log in with email/password or Google OAuth	High	`Module/Identity/Features/Store/Auth/Login/Password/`, `Shared/Security/Authentication/External/`
ID-FR-03	JWT access tokens (15-minute expiry) with refresh-token rotation and reuse detection	High	`appsettings.json:30-43`, `Shared/Security/Authentication/Tokens/Services/Refresh/`
ID-FR-04	Guest sessions via cookie for anonymous cart usage	High	`appsettings.json:88-94`, `Module/Ordering/Features/Storefront/Cart/AssociateCart/`
ID-FR-05	Role-based and permission-based authorization; admin vs storefront surface segregation	High	`Shared/Security/Authorization/Registry/PermissionContext.cs`, `Shared/Security/Authorization/Attributes/HasPermission.Attribute.Ext`
ID-FR-06	Password reset via email token	Medium	`Module/Identity/Features/Store/Passwords/`
ID-FR-07	Administrators can manage users, roles, and permissions	High	`Module/Identity/Features/Admin/Users/`, `Module/Identity/Features/Admin/Roles/`, `Module/Identity/Features/Admin/Permissions/`

2.1.3 Inventory Module

Table 7.

Table 7: Inventory Module Functional Requirements

ID	Requirement	Priority	Evidence
INV-FR-01	Stock locations (warehouses) can be created and managed	High	`Module/Inventory/Domain/StockLocations/StockLocation.cs`
INV-FR-02	Stock items track quantity on hand per variant per location	High	`Module/Inventory/Domain/StockLocations/StockItems/StockItem.cs`
INV-FR-03	Stock reservations hold inventory for active carts/orders	High	`Module/Inventory/Domain/StockReservations/StockReservation.cs`
INV-FR-04	Stock transfers move inventory between locations	Medium	`Module/Inventory/Domain/StockTransfers/StockTransfer.cs`
INV-FR-05	Stock movements (adjustments) are auditable	Medium	`Module/Inventory/Domain/StockLocations/StockItems/StockMovements/StockMovement.cs`

2.1.4 Ordering Module

Table 8.

Table 8: Ordering Module Functional Requirements

ID	Requirement	Priority	Evidence
ORD-FR-01	Guest and authenticated users can add items to a cart	High	`Module/Ordering/Features/Storefront/Cart/AddItem/`
ORD-FR-02	Carts auto-expire after 7 days of inactivity (Hangfire background job)	Medium	`appsettings.json:181-183`, `Module/Ordering/Backgrounds/CartExpiryJob.cs`
ORD-FR-03	Checkout proceeds through states: Address → Delivery → Payment → Confirm → Complete	High	`Module/Ordering/Domain/Orders/Order.cs:20`, `Order.Constant.cs:50-56`
ORD-FR-04	Orders calculate totals: ItemTotal + AdjustmentTotal + ShipmentTotal = Total	High	`Module/Ordering/Domain/Orders/Order.cs:22-25`, invariant comment line 12
ORD-FR-05	Orders track payment state and shipment state independently	High	`Module/Ordering/Domain/Orders/Order.cs:28-29`
ORD-FR-06	Orders can be canceled (with reason: customer or admin)	High	`Module/Ordering/Features/Storefront/Orders/Cancel/`, `Module/Ordering/Features/Admin/Orders/Cancel/`
ORD-FR-07	Guest carts can be associated with a user upon login/registration	High	`Module/Ordering/Features/Storefront/Cart/AssociateCart/`
ORD-FR-08	Order number generation inside database transaction with RepeatableRead isolation	High	`git log: commit 887a77c7`

2.1.5 Payment Module

Table 9.

Table 9: Payment Module Functional Requirements

ID	Requirement	Priority	Evidence
PAY-FR-01	Payment intents can be created for an order	High	`Module/Payment/Features/Storefront/Payment/CreateIntent/`
PAY-FR-02	Payment intents can be confirmed (capture funds)	High	`Module/Payment/Features/Storefront/Payment/Confirm/`
PAY-FR-03	Administrators can capture, void, and refund payments	High	`Module/Payment/Features/Admin/Payments/Capture/`, `Void/`, `Refund/`
PAY-FR-04	Stripe webhook handling with signature validation	High	`Module/Payment/Features/Storefront/Payment/Webhooks/StripeWebhook.cs:32-36`
PAY-FR-05	Bogus gateway for development/testing	High	`Module/Payment/Services/Provider/Bogus/BogusGateway.cs`

2.1.6 Shipping Module

Table 10.

Table 10: Shipping Module Functional Requirements

ID	Requirement	Priority	Evidence
SHIP-FR-01	Shipping methods can be configured (name, carrier, zones)	High	`Module/Shipping/Domain/ShippingMethods/ShippingMethod.cs`
SHIP-FR-02	Shipping rates calculated based on address and method	High	`Module/Shipping/Domain/ShippingRates/ShippingRate.cs`, `Calculators/ShippingRateCalculator.cs`

2.1.7 Profile Module

Table 11.

Table 11: Profile Module Functional Requirements

ID	Requirement	Priority	Evidence
PROF-FR-01	Users manage profile, addresses, wishlists	High	`Module/Profile/Domain/UserProfile.cs`, `Addresses/Address.cs`, `Wishlists/Wishlist.cs`
PROF-FR-02	Notification preferences per channel (email, SMS)	Medium	`Module/Profile/Domain/Notifications/NotificationPreference`

2.1.8 Location Module

Table 12.

Table 12: Location Module Functional Requirements

ID	Requirement	Priority	Evidence
LOC-FR-01	Countries and states/provinces with ISO codes	High	`Module/Location/Features/Admin/Countries/`, `Module/Location/Features/Admin/States/`

2.2 NON-FUNCTIONAL REQUIREMENTS

Table 13.

Table 13: Non-Functional Requirements

ID	Requirement	Target	Evidence
NFR-01	**Modularity** — Modules must have zero direct cross-references	Compile-time isolation	`AGENTS.md:10`, `Directory.Build.targets:`
NFR-02	**Explicit error handling** — No exceptions for control flow; all failures return `Result<T>`	100% of handlers	`AGENTS.md:10`, `Result.cs:1-43`
NFR-03	**Build strictness** — Warnings treated as errors	Zero warnings	`Directory.Build.props:17`
NFR-04	**Testability** — Unit tests runnable without Docker; integration tests with Docker	xUnit v3, Testcontainers	`Directory.Packages.props`
NFR-05	**Observability** — OpenTelemetry traces, metrics, logs; correlation IDs	OTLP export optional	`infra/Aspire/src/ReSys.ServiceDefaultExtensions.cs:58-103`
NFR-06	**Rate limiting** — Named policies per endpoint category	auth: 5/min, register: 3/hr, payment: 30/min	`appsettings.json:79-86`
NFR-07	**Security headers** — CSP, HSTS, X-Frame-Options, etc.	All responses	`Shared/Security/Headers/SecurityHeadersM`
NFR-08	**File upload security** — Magic-byte validation, extension allowlist, size limit, anti-forgery	Max 10 MB; 15 blocked extensions	`appsettings.json:129-155`
NFR-09	**Background jobs** — Hangfire with Redis	Job survives restart (Redis)	`Shared/Operational/Backgrounds/Background.Extension.cs:104-122`

2.3 BUSINESS RULES

Business rules are encoded as domain invariants (comments on entity classes), validation rules (FluentValidation), and domain methods (factory methods, state transitions).

Table 14.

Table 14: Business Rules

Rule	Type	Enforcement	Evidence
Product slug must be unique	Invariant	EF query in `CreateProduct` handler + db unique index	`CreateProduct.cs:41-43`
At least one `OptionType` if `Product.HasVariants`	Invariant	Domain check	`Product.cs` invariant comment line 14
Master variant must exist per product	Invariant	`CreateProduct` dispatches `AddVariant` before finalizing	`CreateProduct.cs:64-65`
Order total = ItemTotal + AdjustmentTotal + ShipmentTotal	Invariant	Domain property calculation	`Order.cs:22-25`, comment line 12
Finalized orders are immutable except Cancel	Invariant	`Order` state machine	`Order.cs` comment line 12
Cart expires after 7 days	Policy	Hangfire `CartExpiryJob`	`CartExpiryJob.cs`, `appsettings.jso`
Refresh token reuse triggers blacklist	Security	`ReuseDetectionEnabled=true`	`appsettings.json:40`
Payment signature must be validated before processing	Security	Stripe webhook handler	`StripeWebhook.cs:32-36`

2.4 USE CASES

2.4.1 Actor–Use Case Mapping

Table 15.

Table 15: Actor–Use Case Mapping

Actor	Use Cases
Customer (Guest / Authenticated)	UC-CAT-01: Browse products UC-CAT-02: Search by image UC-CAT-03: View product details UC-ORD-01: Add to cart UC-ORD-02: Checkout UC-ORD-03: View order history UC-ORD-04: Cancel order UC-ID-01: Register / Login UC-ID-02: Login with Google UC-ID-03: Reset password UC-PROF-01: Manage addresses UC-PROF-02: Manage wishlist UC-PROF-03: Set notification preferences UC-PAY-01: Create payment intent UC-PAY-02: Confirm payment UC-SHIP-01: Calculate shipping
Administrator	UC-CAT-04: Manage products UC-CAT-05: Manage taxonomies UC-CAT-06: Upload variant images UC-ORD-05: Manage orders UC-ID-04: Manage users & roles UC-PAY-03: Refund / capture UC-SHIP-02: Configure methods
System (Background Jobs)	UC-ORD-06: Expire abandoned carts UC-PAY-04: Handle Stripe webhook

2.4.2 Use Case: Search by Image (CBIR)

Actor: Storefront Customer
Precondition: Product catalog contains images with embeddings (generated by currently configured model)
Flow:

1. Customer uploads an image via Storefront SPA
2. Storefront POSTs to `/api/catalog/storefront/search-by-image`
3. Backend forwards image bytes to Python embedding sidecar (`/embeddings`)
4. Sidecar loads the configured embedding model (e.g., Fashion-CLIP, ResNet-50) and generates a vector
5. Backend queries PostgreSQL pgvector with cosine similarity: `SELECT * FROM variant_images ORDER BY embedding <=> $1 LIMIT 20` (embeddings filtered by `model_name`)
6. Results mapped to DTOs and returned

Postcondition: Customer receives visually similar products

Evidence: `ApiTests/Catalog/Storefront/search-by-image.http`, `ImageEmbedding.Inference.cs:21-36`, `Vector.Configuration.cs`

2.4.3 Use Case: Model Comparison Evaluation (Research)

Actor: System / Researcher
Precondition: Ground-truth dataset of 100 fashion images with human-labeled similarity groups exists
Flow:
1. Configure sidecar with Model A (e.g., Fashion-CLIP)
2. Generate embeddings for all 100 query images and all catalog images
3. Execute top-20 similarity search for each query image
4. Record Precision at K=20, Recall at K=20, embedding generation time per image, total storage
5. Repeat steps 1–4 for Model B (ResNet-50), Model C (EfficientNet-B0), Model D (CLIP-generic)
6. Compute mean ± SD across all queries per model
7. Generate comparison report: retrieval metrics vs. operational metrics trade-off

Postcondition: Identification of optimal model for deployment based on empirical evidence

Evidence: `11-evaluation.md:§11.5`

2.4.4 Use Case: Checkout

Actor: Authenticated or Guest Customer
Precondition: Cart contains items; stock is available
Flow:
1. Customer sets address → `CheckoutState = Address`
2. Customer selects shipping method → `CheckoutState = Delivery`
3. Customer selects payment method → `CheckoutState = Payment`
4. Backend creates payment intent (Stripe or Bogus)
5. Customer confirms → `CheckoutState = Confirm`
6. Backend finalizes: creates order, deducts stock, clears cart → `CheckoutState = Complete`

Postcondition: Order created with `Status = Draft` (or `Pending`); payment intent linked

Evidence: `Module/Ordering/Features/Storefront/Cart/Checkout/CreateOrderFromCart.cs`, `Order.Constant.cs:50-56`

2.5 USER ROLES

Table 16.

Table 16: User Roles and Permissions

Role	Permissions	Surfaces
Guest	Browse catalog, add to cart, search by image, checkout	Storefront
Customer	All guest actions + profile, addresses, wishlists, order history	Storefront
Admin	Full CRUD on all modules, user management, payment captures/refunds, shipping configuration	Admin SPA
System	Background jobs, webhook handlers, cart expiry	Backend only

2.6 EVIDENCE

- `service/Api/src/Api/appsettings.json:1-237` — runtime configuration documenting all functional areas
- `service/Api/src/Module/*/Domain/*/*.cs` — domain entities with invariant comments
- `ApiTests/` — 49 `.http` files documenting all endpoints (use case realization)
- `service/Api/src/Shared/Security/Authorization/Registry/PermissionContext.cs:1-60` — permission enumeration per role
- `service/Api/src/Module/Catalog/Features/Admin/Products/Create/CreateProduct.cs:36-78` — concrete feature implementation
- `service/Api/src/Module/Ordering/Features/Storefront/Cart/Checkout/CreateOrderFromCart.cs` — checkout flow

3 CHAPTER 2: SYSTEM ARCHITECTURE

3.1 ARCHITECTURAL STYLE

3.1.1 Decision: Modular Monolith

Decision: Deploy the entire backend as a single ASP.NET Core process (`service/Api/src/Api`) containing 8 business modules within one assembly (`Module.csproj`).

Alternatives considered:

Table 17.

Table 17: Architectural Style Alternatives

Alternative	Pros	Cons	Why rejected
Microservices	Independent deployability, per-module scaling	Operational complexity (service mesh, distributed tracing, eventual consistency for checkout), overkill for thesis scope	Thesis requires demonstrability and simplicity over operational scale
Clean Architecture (per-module assemblies)	Strict layer boundaries via compiler	Slower builds, excessive project count (8 modules × 4 layers = 32 projects), references still cross-cut	Single assembly with namespace isolation is sufficient for a single team
Modular Monolith	Single deployable, in-process consistency, easier debugging, testable module isolation	Cannot scale modules independently	Fits the thesis constraint of "demonstrable single system" while keeping modules logically isolated

Justification: A modular monolith gives us the compile-time boundaries of modular design (no cross-module references) without the operational overhead of microservices. All 8 modules share one `ApplicationDbContext` and one transaction boundary, which simplifies the checkout flow (order + payment + inventory update can be ACID). The trade-off — inability to scale modules in-

dependently — is acceptable because the thesis evaluates architectural process, not production operational scale.

Rationale in depth:

The rejection of microservices is grounded in the observation that distributed systems introduce **accidental complexity** (Brooks, 1986) that overshadows the essential complexity of the problem. For a single-developer thesis project, the operational burden of service discovery, distributed tracing, sagas for multi-service transactions, and independent deployment pipelines would consume the majority of the available time — leaving insufficient capacity for the actual research contribution (CBIR integration and explicit error handling). As Newman (**Monolith to Microservices**, 2019) argues, microservices are a solution to organizational scale (Conway's Law), not technical scale. A single team does not experience the coordination friction that microservices are designed to solve.

The rejection of per-module Clean Architecture assemblies is similarly pragmatic. While 32 projects (8 modules × 4 layers) would enforce strict compile-time boundaries, the build overhead and reference management would dominate the development workflow. In a thesis timeline, the cost of waiting for incremental builds and resolving circular references across 32 `.csproj` files is not justified by the benefit. Namespace isolation within a single assembly, combined with the `ValidateVerticalSliceIsolation` target (intention, even if currently disabled), provides sufficient boundary enforcement for a demonstrable system.

The modular monolith, therefore, is not a compromise — it is a **conscious architectural decision** that optimizes for the thesis constraints: demonstrability, single-team development, ACID consistency for checkout, and sufficient module isolation to evaluate the design patterns under study.

Evidence: `service/Api/src/Api/Program.cs:38-45` (8 `AddXxxModule()` calls), `service/Api/src/Module/Module.csproj:1-21` (single assembly)

3.1.2 Decision: Vertical Slice Architecture

Decision: Organize code by *feature* rather than by *technical layer*. Each feature action lives in `Features/\{Admin\|Storefront\}/\{Feature\}/\{Action\}/` as a `static partial class` split across 5 files: Handler, Endpoint, Request, Response, and Validator.

Justification: Traditional horizontal layering (Controllers/Services/Repositories) scatters a single use case across the codebase. Vertical slicing makes each use case self-contained: a reviewer can understand "Create Product" entirely by reading one folder. This is critical for thesis evaluation because

examiners can trace a requirement directly to its implementation without cross-referencing multiple layers.

Evidence: `Module/Catalog/Features/Admin/Products/Create/CreateProduct.cs`, `CreateProduct.Endpoint.cs`, `CreateProduct.Request.cs`, `CreateProduct.Response.cs`, `CreateProduct.Validator.cs`

3.1.3 Decision: CQRS via MediatR

Decision: Separate commands (mutations) from queries (reads) using MediatR `ICommand<TResponse>` and `IQuery<TResponse>` contracts. All feature handlers implement these interfaces.

Justification: CQRS decouples the transport layer (Carter minimal API endpoint) from the business logic. More importantly, it enables *pipeline behaviors* — cross-cutting concerns (logging, validation, exception mapping) that wrap every request without polluting handlers. This demonstrates the Decorator pattern in practice.

Evidence: `Shared/Application/Mediators/Commands/ICommand.cs`, `Shared/Application/Mediators/Queries/IQuery.cs`, `Shared/Application/Mediators/Mediator.Extension.cs:46-50` (pipeline registration)

3.1.4 Decision: Result Objects (Not Exceptions)

Decision: All domain and handler operations return `Result<T>` or `Result`. Exceptions are reserved for unrecoverable infrastructure failures only.

Justification: Exception-driven control flow hides error paths in implicit stack unwinding. `Result<T>` makes every failure path explicit, type-safe, and testable. This directly addresses the thesis objective of "predictable error handling." The design follows Railway-Oriented Programming principles.

Evidence: `Shared/Application/Models/Results/Result.cs:1-43`, `Result.Method.cs:84-152` (factory methods: `Result.NotFound`, `Result.Conflict`, `Result.Validation`)

3.2 SYSTEM CONTEXT

The C4 Context diagram describes the system's boundaries and external actors. Three primary actors interact with ReSys.Shop:

- **Customer (Storefront)**: Browses the catalog, searches by image, manages cart, and completes checkout via the Storefront SPA.

- **Administrator (Admin)**: Manages products, inventory, orders, users, and system configuration via the Admin SPA.
- **System (Webhooks)**: External services (e.g., Stripe) push events to the backend via webhook endpoints.

The backend is a single ASP.NET Core API (.NET 10) exposing REST endpoints under `/api`. It depends on:

- **PostgreSQL 17 + pgvector**: Primary relational database with vector similarity search support.
- **Redis 7**: Caching (HybridCache L2) and Hangfire job storage.
- **Python ML Sidecar**: FastAPI service running Fashion-CLIP and other embedding models for CBIR.

External integrations include Stripe (payments), SendGrid/SMTP (email), Sinch (SMS), Google OAuth (login), and S3 (file storage).

3.3 CONTAINER DIAGRAM

The C4 Container diagram details the runtime components:

- **Store SPA** (Vue 3 + Nuxt UI, port 5174): Customer-facing storefront application.
- **Admin SPA** (Vue 3 + PrimeVue, port 5173): Internal administration dashboard.
- Both SPAs communicate with the backend via HTTP REST API calls to `/api`.

The **ASP.NET Core API** (port 5035) is the central container, hosting:

- **Carter endpoints** (minimal API routing)
- **MediatR pipeline** (Logging → Validation → ExceptionMapping behaviors)
- **8 Module handlers** + Shared infrastructure

Downstream containers:

- **PostgreSQL 17 with pgvector**: Stores all relational data and vector embeddings.
- **Redis 7**: HybridCache and Hangfire persistence.
- **Embedding Sidecar** (port 8000): Python FastAPI service for image vector generation.
- **Stripe Gateway**: Webhook-based payment processing.

3.4 COMPONENT DIAGRAM (API BACKEND)

The C4 Component diagram reveals the internal architecture of the API host:

MediatR Pipeline (request processing chain):

1. `LoggingBehavior` — logs entry with CorrelationId

2. `ValidationBehavior` — FluentValidation short-circuit on errors
3. `ExceptionMappingBehavior` — catches exceptions, maps to `Result.Unexpected`

Requests flow through the pipeline to one of three handler types:

- **Commands** (mutations): Create, Update, Delete operations
- **Queries** (reads): Get, List, Search operations
- **IPagedQuery** (paginated): Paginated list operations

All handlers interact with:

- **ApplicationDbContext** (EF Core) with interceptors: `AuditableInterceptor`, `SoftDeletableInterceptor`, `VersionableInterceptor`, and Specification DSL.

Cross-cutting concerns are provided as separate components:

- **Security**: JWT + OAuth authentication
- **Storage**: Local/S3 file storage (Strategy pattern)
- **Notifications**: Email + SMS (SendGrid/SMTP/Sinch)
- **Backgrounds**: Hangfire job processing

3.5 DESIGN PATTERNS

Table 18.

Table 18: Design Patterns

Pattern	Location	Justification
Vertical Slice	Every `Features/\{Admin|Storefront\}/\{Feature\}/\{Action\}/`	Cohesion: all code for one use case in one place
CQRS	`ICommand<>`, `IQuery<>`, separate handlers	Read/write optimization; pipeline behaviors
Pipeline (Decorator)	`LoggingBehavior → ValidationBehavior → ExceptionMappingBehavior`	Cross-cutting concerns without AOP frameworks
Result / Railway	`Result<T>`, `Error`	Explicit control flow; compiler-enforced error handling
Module Isolation	`AddXxxModule()` extension methods; no `Module.X → Module.Y` refs	Independent reasoning about each domain
Strategy	`IStorageProvider` (Local/ S3), `IGatewayRegistry` (Stripe/Bogus), `INotificationService` (SendGrid/SMTP/Sinch)	Pluggable providers without touching call sites
Options + FluentValidation	Every settings type has a validator	Fail-fast configuration errors at boot
Specification	`Shared/Operational/Persistence/Specifications/`	Composable query expressions; declarative filtering
Repository (Pragmatic)	`IApplicationDbContext` as unit-of-work; `DbSet<T>` queried directly	Avoids unnecessary abstraction; EF Core + interceptors suffice
Factory	Domain entity constructors are `internal`; creation via `MapToDomain()` / factory methods	Enforces invariants at creation time

3.6 DATA FLOW

3.6.1 Normal Request Flow

The standard request lifecycle follows this sequence:

```
HTTP Request
  → Carter Endpoint (ICarterModule.Map, AddEndpoints scans
assemblies)
    → Endpoint calls sender.Send(new Command(request))
      → LoggingBehavior (log entry with CorrelationId)
        → ValidationBehavior (FluentValidation; short-circuit on
errors → Result.Validation)
          → ExceptionMappingBehavior (try/catch →
Result.Unexpected)
            → Command/Query Handler
              → Domain logic (factory methods, invariants)
              → EF Core SaveChanges / external API call
              → Mapster mapping to Response DTO
    → result.ToResult() → IResult with status code + JSON
envelope
  → HTTP Response
```

Evidence: `Program.cs:54-65`, `Mediator.Extension.cs:46-50`, `Validation.Behavior.cs:1-67`, `Exception.Behavior.cs:1-42`

3.6.2 Image Search Flow (CBIR — Model-Agnostic)

The image search flow enables customers to find visually similar products:

```
User uploads image
  → Storefront SPA POST /api/catalog/storefront/search-by-image
    → Backend receives image bytes
      → HTTP POST to Python sidecar /embeddings (Aspire service
discovery)
        → Sidecar loads configured model (Fashion-CLIP /
ResNet-50 / EfficientNet-B0 / CLIP-generic)
        → Sidecar generates vector (dimension varies: 512, 2048,
1280, 512)
      → Backend receives vector + model_name
        → EF Core + pgvector: cosine similarity search filtered
by model_name
      → Mapster maps results to Product DTOs
    → JSON response with similar products
```

Model abstraction: The sidecar exposes `POST /embeddings` with a configurable `model` parameter (default from env var `EMBEDDING_MODEL`). Each model implements `BaseEmbeddingModel` with `encode_image()` → `np.ndarray`. The database stores `model_name` alongside each embedding to enable per-model indexing and comparison.

Evidence: `ImageEmbedding.Inference.cs:21-36`, `Vector.Configuration.cs:1-30+`, `ApiTests/Catalog/Storefront/search-by-image.http`

3.6.3 Model Comparison Flow (Evaluation)

The evaluation flow benchmarks multiple embedding models against a ground-truth dataset:

```
Ground-truth dataset (100 images, 10 similarity groups)
  → For each model in [Fashion-CLIP, ResNet-50, EfficientNet-B0,
CLIP-generic]:
    → Configure sidecar: EMBEDDING_MODEL=<model_name>
    → Restart sidecar (model swap)
    → Generate embeddings for all catalog images
    → For each query image in ground-truth:
      → POST /embeddings → receive vector
      → Query pgvector top-20
      → Compare retrieved variants against labeled group
      → Record: Precision at K=20, Recall at K=20, latency_ms,
vector_dim
    → Compute mean ± SD across 100 queries
  → Generate comparison table
```

Evidence: `11-evaluation.md:§11.5`

3.6.4 Checkout Flow (Critical Path)

The checkout flow is the most critical data path, requiring ACID consistency across order creation, payment intent, and inventory reservation:

```
Cart items present
  → POST /api/ordering/storefront/cart/checkout
    → CreateOrderFromCart handler
      → Validate cart not empty, items in stock
      → Generate order number inside DB transaction
(RepeatableRead)
      → Create Order entity with line items
      → Create Payment Intent via gateway (Stripe/Bogus)
      → SaveChanges (Order + PaymentIntent)
    → Return Order DTO with payment client secret
```

Evidence: `CreateOrderFromCart.cs`, `Order.cs`, `PaymentIntent.cs`, git log: commits `887a77c7`, `bd042088`

3.7 EVIDENCE

- `service/Api/src/Api/Program.cs:1-66` — composition root and module wiring
- `service/Api/src/Shared/Application/Mediators/Mediator.Extension.cs:1-79` — MediatR + pipeline behaviors
- `service/Api/src/Shared/Application/Models/Results/Result.cs:1-43`, `Result.Method.cs:1-191` — Result pattern
- `service/Api/src/Module/Catalog/Features/Admin/Products/Create/*.cs` — vertical slice anatomy
- `service/Api/src/Shared/Operational/Storages/Storage.Extensions.cs:1-115` — Strategy pattern for storage
- `infra/Aspire/src/ReSys.AppHost/AppHost.cs:1-49` — Aspire orchestration wiring
- `service/Embedding/src/main.py:1-29` — Python sidecar entry

4 CHAPTER 3: DOMAIN DESIGN

4.1 DOMAIN MODEL OVERVIEW

ReSys.Shop implements a **Domain-Driven Design (DDD)** approach with the following characteristics:

- **Aggregates** are the consistency boundary; each aggregate root is an entity with a unique identity
- **Entities** have lifecycle continuity (e.g., `Order`, `Product`, `Variant`)
- **Value Objects** are immutable and equality-based (e.g., `Money` concepts represented as `decimal` with currency string)
- **Domain Services** are stateless operations that don't belong to an entity (e.g., `ShippingRateCalculator`)
- **Domain Events** are raised for significant state changes (e.g., `OrderPlacedEvent` — though the current publisher infrastructure is in flux, see CONCERNS.md)

Design decision: The project pragmatically avoids over-engineering DDD patterns. There are no explicit `ValueObject` base classes; instead, value semantics are modeled via records or primitive types with validation. The aggregate boundaries are enforced through EF Core navigation properties and handler-level transaction control, not through event sourcing or complex repository patterns.

Evidence: `AGENTS.md:10-12`, `Module/*/Domain/*/*.cs`, `Shared/Application/Domain/Models/Entity.cs`

4.2 BOUNDED CONTEXT MAP

ReSys.Shop is decomposed into **8 bounded contexts**, each corresponding to one business module. A formal Context Map diagram is maintained at `docs/thesis/diagrams/bounded-context-map.mmd`.

Integration pattern: All contexts integrate via **in-process message dispatch** (MediatR `ISender`). There are no direct namespace references between contexts — this is the **Conformist** pattern with a shared technical kernel (`Shared.Application` containing `Result<T>`, `ICommand`, `IQuery`).

Table 19.

Table 19: Bounded Context Map

Context	Responsibilities	Published Language (shared terms)
Catalog	Products, variants, images, taxonomies, option types, classifications	`ProductId`, `VariantId`, `Sku`, `Price`, `Slug`
Ordering	Carts, orders, line items, adjustments, checkout state machine	`OrderId`, `OrderNumber`, `Total`, `Currency`, `CheckoutState`
Payment	Payment intents, captures, refunds, voids, Stripe webhooks	`PaymentIntentId`, `PaymentState`, `ClientSecret`
Inventory	Stock locations, items, reservations, transfers, movements	`StockItemId`, `QuantityOnHand`, `StockLocationId`
Identity	Users, roles, permissions, JWT tokens, OAuth, guest sessions	`UserId`, `Email`, `PermissionClaim`
Profile	Addresses, wishlists, notification preferences	`ProfileId`, `AddressId`, `WishlistId`
Shipping	Shipping methods, rates, calculators, zones	`ShippingMethodId`, `Rate`, `Zone`
Location	Countries, states, ISO codes	`CountryId`, `StateId`, `IsoCode`

Design rationale: Using a modular monolith with 8 namespaces in one assembly means the bounded contexts share a common **technical kernel** (`Shared.Application`). This is a pragmatic trade-off: we get compile-time type safety and easy refactoring across contexts, while still maintaining logical boundaries through convention and the `ValidateVerticalSliceIsolation` target (currently disabled, but documented in `Directory.Build.targets:42-53`).

Evidence: `AGENTS.md:10-12`, `Module/*/Domain/`, `Shared/Application/`, `Directory.Build.targets:42-53`

4.3 AGGREGATE BOUNDARIES

An aggregate is a cluster of associated objects treated as a single unit for data changes. Each aggregate has one root entity.

4.3.1 Catalog Aggregates

Table 20.

Table 20: Catalog Aggregates

Aggregate	Root	Children / Value Objects	Invariants
Product	`Product`	`Variant` (1..n), `ProductOptionType` (0..n), `Classification` (0..n)	Slug unique; `HasVariants` → ≥1 `OptionType`; `MasterVariantId` must exist
Variant	`Variant`	`Price` (1..n), `OptionValueVariant` (0..n), `VariantImage` (0..n)	SKU unique; `IsMaster` exclusive per Product; `TrackInventory` → StockItem exists
Taxonomy	`Taxonomy`	`Taxon` (tree structure)	Taxon tree integrity
OptionType	`OptionType`	`OptionValue` (1..n)	Values required

Evidence: `Module/Catalog/Domain/Products/Product.cs`, `Module/Catalog/Domain/Products/Variants/Variant.cs`, `Module/Catalog/Persistence/CatalogSchema.cs:1-31`

4.3.2 Ordering Aggregates

Table 21.

Table 21: Ordering Aggregates

Aggregate	Root	Children / Value Objects	Invariants
Order	`Order`	`LineItem` (1..n), `Adjustment` (0..n)	`Total = ItemTotal + AdjustmentTotal + ShipmentTotal`; checkout state advances forward; finalized orders immutable except Cancel
Cart	(implicit — `Order` with `Status = Draft` and `SessionId`)	Same as Order	Expires after 7 days

Evidence: `Module/Ordering/Domain/Orders/Order.cs`, `Order.Constant.cs:1-98`

4.3.3 Payment Aggregates

Table 22.

Table 22: Payment Aggregates

Aggregate	Root	Children	Invariants
PaymentIntent	`PaymentIntent`	`PaymentCapture` (0..n)	State machine: `Pending` → `RequiresAction` → `Processing` → `Succeeded` / `Canceled`

Evidence: `Module/Payment/Domain/PaymentCaptures/PaymentCapture.cs`, `PaymentCapture.Method.State.cs`

4.3.4 Identity Aggregates

Table 23.

Table 23: Identity Aggregates

Aggregate	Root	Children	Invariants
User	`User` (extends `IdentityUser<Guid>`)	`UserClaim`, `UserRole`, `UserLogin`, `UserToken`, `UserPasskey`	Email uniqueness enforced by ASP.NET Identity
Role	`Role`	`RoleClaim`	Permission claims stored as claims

Evidence: `Shared/Security/Identity/Domain/Users/User.cs`, `Shared/Security/Identity/Domain/Roles/Role.cs`

4.3.5 Inventory Aggregates

Table 24.

Table 24: Inventory Aggregates

Aggregate	Root	Children	Invariants
StockLocation	`StockLocation`	`StockItem` (1..n)	Location must be active to hold stock
StockItem	`StockItem`	`StockMovement` (0..n)	Quantity cannot go negative (enforced by domain method)
StockReservation	`StockReservation`	—	Linked to cart/order; auto-expires
StockTransfer	`StockTransfer`	—	Source and destination must be different

Evidence: `Module/Inventory/Domain/StockLocations/StockLocation.cs`, `Module/Inventory/Domain/StockLocations/StockItems/StockItem.cs`

4.3.6 Profile Aggregates

Table 25.

Table 25: Profile Aggregates

Aggregate	Root	Children	Invariants
UserProfile	`UserProfile`	`Address` (0..n), `Wishlist` (0..1)	One profile per user
Wishlist	`Wishlist`	`WishedItem` (0..n)	Product + Variant uniqueness within list
NotificationPreferences	`NotificationPreferences`	—	Per-channel enablement flags

Evidence: `Module/Profile/Domain/UserProfile.cs`, `Module/Profile/Domain/Wishlists/Wishlist.cs`

4.3.7 Shipping Aggregates

Table 26.

Table 26: Shipping Aggregates

Aggregate	Root	Children	Invariants
ShippingMethod	`ShippingMethod`	`ShippingRate` (1..n)	Method must have at least one rate

Evidence: `Module/Shipping/Domain/ShippingMethods/ShippingMethod.cs`, `Module/Shipping/Domain/ShippingRates/ShippingRate.cs`

4.4 ENTITIES AND VALUE OBJECTS

4.4.1 Base Entity Design

All domain entities inherit from `Entity` (defined in `Shared.Application.Domain.Models`):

```
public abstract class Entity
{
    public Guid Id { get; protected set; }
}
```

Design rationale: Using `Guid` as the primary key type provides:

- Natural distributed ID generation (no central sequence required)
- Security through obscurity (sequential IDs leak business volume)
- Easier data merging across environments

Trade-off: GUIDs are larger than integers (16 bytes vs 4-8 bytes) and fragment clustered indexes. Mitigation: The project uses PostgreSQL which handles UUID indexes efficiently; composite indexes on `(UserId, Status)` and `(SessionId, Status)` are explicitly added for query performance (`git log: commit bd042088`).

Evidence: `Shared/Application/Domain/Models/Entity.cs`, `Order.cs:14` (inherits `Entity`)

4.4.2 Cross-Cutting Domain Concerns

The project uses interface markers for cross-cutting behavior rather than base classes (to avoid diamond inheritance):

Table 27.

Table 27: Cross-Cutting Domain Concerns

Concern	Interface	Applied to
Auditable	`IAuditable`	All business entities
Soft-deletable	`ISoftDeletable`	All business entities
Versionable	`IVersionable`	Entities requiring optimistic concurrency

Evidence: `Shared/Application/Domain/Concerns/Auditable/IAuditable.cs`, `SoftDeletable/ISoftDeletable.cs`, `Shared/Operational/Persistence/Interceptors/Auditable.Interceptor.cs`

4.5 STATE MACHINE DIAGRAMS

4.5.1 Order Lifecycle (CheckoutState)

Table 28.

Table 28: Order Checkout State Transitions

From	Trigger	To
start	Create cart / start checkout	Address
Address	Set shipping address	Delivery
Address	Cancel	end
Delivery	Select shipping method	Payment
Delivery	Cancel	end
Payment	Select payment method	Confirm
Payment	Cancel	end
Confirm	Confirm order	Complete
Confirm	Cancel	end
Complete	Order finalized	end

The order checkout state machine progresses through five states: **Address** (initial) → **Delivery** → **Payment** → **Confirm** → **Complete**. State transitions are monotonic (no backward transitions except cancellation of the entire order).

Business rule: CheckoutState progresses forward; finalized orders are immutable except Cancel (`Order.cs:12`).

Evidence: `Order.cs:20` (`CheckoutState` property), `Order.Constant.cs:50-56`

4.5.2 Payment Intent Lifecycle

Table 29.

Table 29: Payment Intent State Transitions

From	Trigger	To
start	Create intent	Pending
Pending	3D Secure / SCA required	RequiresAction
Pending	Payment method attached	Processing
Pending	Cancel intent	Canceled
RequiresAction	Customer authenticates	Processing
RequiresAction	Authentication fails	Canceled
Processing	Charge succeeds	Succeeded
Processing	Charge fails	Failed
Succeeded	Capture funds	Captured
Succeeded	Cancel (before capture)	Canceled
Captured	Refund payment	Refunded
Captured	Fulfillment complete	end
Failed	Retry or abandon	end
Canceled	Order canceled	end
Refunded	Money returned	end

The payment intent state machine tracks the lifecycle of a payment through these states. Each state transition is driven by webhook events from the payment gateway (Stripe) or by explicit gateway API calls.

Design decision: The system maintains its own `PaymentIntent` entity state in parallel with Stripe's state to support the Bogus gateway and enable offline operations.

Evidence: `PaymentCapture.Method.State.cs`, `StripeWebhook.cs`

4.6 BUSINESS RULES (DOMAIN INVARIANTS)

Business rules are enforced at three levels:

Table 30.

Table 30: Business Rule Enforcement Levels

Level	Mechanism	Example
Domain entity	Property setters / factory methods	`Order.Total` is computed, not set directly
Domain method	`Order.Method.Checkout.cs`, `Order.Method.Cancel.cs`	Checkout validates prerequisites before state transition
Handler validation	FluentValidation `Validator.cs`	`CreateProduct.Validator.cs` ensures slug format, name length
Database constraint	EF Core configurations (unique indexes, check constraints)	Composite index on `(UserId, Status)` for order queries

Evidence: `Module/Ordering/Domain/Orders/Order.Method.Checkout.cs`, `CreateProduct.Validator.cs`, `Shared/Operational/Persistence/Configurations/`

4.7 UBIQUITOUS LANGUAGE GLOSSARY

The following terms have precise, domain-specific meanings within the ReSys.Shop codebase. They are used consistently across code, documentation, and API contracts.

Table 31.

Table 31: Ubiquitous Language Glossary

Term	Context	Definition	Code Reference
Aggregate	All	A cluster of domain objects treated as a single unit for data changes; one root entity controls consistency	`Shared/Application/Domain/Models/Entity.cs`
Product	Catalog	A sellable item with name, description, slug, and fashion metadata; may have variants	`Module/Catalog/Domain/Products/Product.cs`
Variant	Catalog	A specific configuration of a product (e.g., Size M + Color Red); has SKU, price, dimensions	`Module/Catalog/Domain/Products/Variants/Variant.cs`
Master Variant	Catalog	The default variant of a product; all products have exactly one master variant	`Product.cs:47`, `Variant.cs:17`
Taxonomy	Catalog	A hierarchical classification system (e.g., "Clothing → Dresses → Evening Dresses")	`Module/Catalog/Domain/Taxonomies/Taxonomy.cs`
Taxon	Catalog	A node in a taxonomy tree; products are classified by association with taxons	`Module/Catalog/Domain/Taxons/Taxon.cs`
Option Type	Catalog	A configurable attribute (e.g., "Size", "Color") with predefined values	`Module/Catalog/Domain/Products/Options/OptionType.cs`
Option Value	Catalog	A specific value of an option type (e.g., "Small", "Red")	`Module/Catalog/Domain/Products/Options/OptionValue.cs`
Order	Ordering	A customer's purchase request; contains line items, totals, checkout state	`Module/Ordering/Domain/Orders/Order.cs`
Line Item	Ordering	A single item within an order (references a variant, quantity, price at time of purchase)	`Module/Ordering/Domain/LineItems/LineItem.cs`
Checkout State	Ordering	The stage of checkout: Address → Delivery → Payment → Confirm → Complete	`Order.cs:20`, `Order.Constant.cs:50`
Cart	Ordering	An implicit aggregate; an Order with [illegible]	`Order.cs:18` (session-based), [illegible]
[illegible]	[illegible]	[illegible]	[illegible]

Design rationale: A Ubiquitous Language glossary prevents "translation friction" between domain experts, developers, and examiners. Every term above is reflected in the codebase as a class name, property, or enum value — ensuring the language is executable, not just documentation.

Evidence: All terms traceable to the domain entity files listed in the Code Reference column.

4.8 EVIDENCE

- `service/Api/src/Module/*/Domain/*/*.cs` — all domain entities
- `service/Api/src/Shared/Application/Domain/Models/Entity.cs` — base entity
- `service/Api/src/Shared/Application/Domain/Concerns/` — cross-cutting domain interfaces
- `service/Api/src/Module/Ordering/Domain/Orders/Order.cs` — aggregate root example
- `service/Api/src/Module/Catalog/Domain/Products/Product.cs` — aggregate root with children
- `service/Api/src/Module/Payment/Domain/PaymentCaptures/PaymentCapture.Method.State.cs` — state machine
- `service/Api/src/Module/Inventory/Domain/StockLocations/StockItems/StockItem.Method.cs` — domain method enforcing invariant

5 CHAPTER 4: DATABASE DESIGN

5.1 DATABASE TECHNOLOGY SELECTION

Decision: PostgreSQL 17 with the `pgvector` extension.

Justification:

Table 32.

Table 32: PostgreSQL + pgvector vs. dedicated vector DB comparison

Criterion	PostgreSQL + pgvector	Alternative (separate vector DB like Pinecone/Milvus)
ACID compliance	Native — checkout + inventory in same transaction	Vector DB is eventually consistent; cross-DB transactions require Saga pattern
Operational complexity	One database to manage, backup, monitor	Two databases + synchronization logic
Query flexibility	SQL + vector similarity in one query (`ORDER BY embedding <=> $1`)	Requires application-level join or dual queries
Cost	Open source, runs locally via Aspire	SaaS pricing or additional self-hosted infrastructure
Thesis demonstrability	`docker run pgvector/pgvector:pg17-trixie`	Extra setup steps for examiners

Trade-off: pgvector's HNSW/IVFFlat indexes are not as optimized as dedicated vector databases for billion-scale vectors. This is acceptable because a thesis catalog contains thousands, not billions, of products.

5.2 NORMALIZATION DESIGN

The database schema is designed in **Third Normal Form (3NF)** with one intentional denormalization. Every table has a single-column surrogate primary key (`uuid id`), no repeating groups, and no transitive dependencies on non-key attributes. Foreign keys reference the surrogate key of their parent table, eliminating update anomalies.

The sole intentional denormalization is the `order.total` column, which stores the pre-computed sum of `item_total + adjustment_total + shipment_total`. This is a **read-optimized denormalization**: the total is recalculated on every write (checkout, refund, adjustment) by the `Order.Method.Checkout.cs` domain method, but stored redundantly to avoid recalculation during the high-frequency read path (order listing, checkout confirmation, admin dashboard). This trade-off is justified by the fact that orders are written far less frequently than they are read, and the recalculation logic is centralized in the domain layer per DDD principles — not scattered in SQL triggers or application queries.

All other computed fields (e.g., `order.item_count`, `variant_image.position`) are maintained by EF Core interceptors or domain methods at write time, preserving 3NF integrity while optimizing read performance.

Evidence: `infra/Aspire/src/ReSys.AppHost/AppHost.cs:11-12`, `Shared/Operational/Persistence/Data/AppDbContext.cs:74-76` (`builder.HasPostgresExtension("vector")`), `Module/Ordering/Domain/Orders/Order.Method.Checkout.cs`

5.3 ENTITY RELATIONSHIP DIAGRAM (ERD)

5.3.1 High-Level ERD (Core Business Entities)

The core business entities form the following relationships:

- **User** (Identity) connects to **Order** via `user_id` foreign key (1:1)
- **Product** connects to **Variant** via `product_id` foreign key (1:n)
- **Order** connects to **LineItem** via `order_id` foreign key (1:n)
- **Variant** connects to **VariantImage** via `variant_id` foreign key (1:n)
- **LineItem** connects to **Variant** via `variant_id` foreign key (1:1)

Key columns per entity:
- **User**: `id` (PK), `email`, `...`
- **Product**: `id` (PK), `name`, `slug` (unique), `master_variant_id`, `...`
- **Order**: `id` (PK), `number`, `status`, `user_id` (FK), `total`, `...`
- **Variant**: `id` (PK), `product_id` (FK), `sku` (unique), `price`, `...`
- **LineItem**: `id` (PK), `order_id` (FK), `variant_id` (FK), `quantity`, `price`
- **VariantImage**: `id` (PK), `variant_id` (FK), `embedding` (pgvector type `vector(512)`), `...`

5.3.2 Identity ERD (ASP.NET Identity Tables)

The Identity schema uses standard ASP.NET Identity table structure:

- **Users** connects to **UserRoles** (1:n), which connects to **Roles** (n:1)
- **Users** connects to **UserClaims** (1:n)
- **Users** connects to **UserLogins** (1:n)
- **Users** connects to **UserTokens** (1:n)
- **Users** connects to **UserPasskeys** (1:n)
- **Roles** connects to **RoleClaims** (1:n)

5.3.3 Profile ERD

- **UserProfile** connects to **Address** (1:n) and **Wishlist** (1:n)
- **Wishlist** connects to **WishedItem** (1:n)
- **WishedItem** references **Variant** via `variant_id`

Key columns:
- **UserProfile**: `id` (PK), `user_id` (unique), `...`

- **Address**: `id` (PK), `profile_id` (FK), `...`
- **Wishlist**: `id` (PK), `profile_id` (FK), `...`
- **WishedItem**: `id` (PK), `wishlist_id` (FK), `variant_id` (FK)

5.3.4 Inventory ERD

- **StockLocation** connects to **StockItem** (1:n)
- **StockItem** connects to **StockMovement** (1:n)

Key columns:
- **StockLocation**: `id` (PK), `name`, `...`
- **StockItem**: `id` (PK), `location_id` (FK), `variant_id` (FK), `quantity`
- **StockMovement**: `id` (PK), `stock_item_id` (FK), `quantity_delta`, `...`

5.4 SCHEMA ORGANIZATION

The database uses **per-module schemas** to provide logical separation within the shared PostgreSQL database:

Table 33.

Table 33: Per-module schema organization

Schema	Tables	Evidence
catalog	products, variants, variant_images, option_types, option_values, taxonomies, taxons, classifications	Module/Catalog/Persistence/CatalogSchema.cs:1-31
ordering	orders, line_items, adjustments	Module/Ordering/Persistence/OrderingSchema.cs
payment	payment_intents, payment_captures, payment_methods	Module/Payment/Persistence/PaymentSchema.cs
inventory	stock_locations, stock_items, stock_movements, stock_reservations, stock_transfers	Module/Inventory/Persistence/InventorySchema.cs
shipping	shipping_methods, shipping_rates	Module/Shipping/Persistence/ShippingSchema.cs
profile	user_profiles, addresses, wishlists, wished_items, notification_preferences	Module/Profile/Persistence/ProfileSchema.cs
location	countries, states	Module/Location/Persistence/LocationSchema.cs
identity	ASP.NET Identity tables (users, roles, user_roles, user_claims, role_claims, user_logins, user_tokens, user_passkeys)	Shared/Security/Identity/Identity.Extension.cs

Design rationale: Schemas provide namespace isolation without the operational cost of separate databases. A DBA can back up or grant permissions per schema. EF Core supports schema mapping via `ToTable("name", "schema")` in entity configurations.

5.5 KEY TABLES AND COLUMNS

5.5.1 Product and Variant

Table 34.

Table 34: Product and variant table details

Table	Key Columns	Constraints	Indexes
catalog.products	id, name, slug, status, master_variant_id, available_on, discontinue_on, style_code, season_name	slug unique	[slug], [status]
catalog.variants	id, product_id, sku, is_master, price, cost_price, track_inventory, weight, height, width, depth	sku unique, FK to products	[product_id], [sku]
catalog.variant_images	id, variant_id, file_path, alt_text, position, embedding, model_name, vector_dim	FK to variants	[variant_id], embedding USING ivfflat (pgvector), [model_name]

5.5.2 Order

Table 35.

Table 35: Order table details

Table	Key Columns	Constraints	Indexes
ordering.orders	id, number, session_id, user_id, store_id, status, checkout_state, currency, item_total, adjustment_total, shipment_total, total, payment_total, outstanding_balance	number unique	**Composite**: (user_id, status), (session_id, status) — added in migration 20260713131410_Ord
ordering.line_items	id, order_id, variant_id, quantity, price, total	FK to orders (NoAction to avoid cascade cycles)	[order_id]

Design decision: The migration 20260713131410_OrderingIndexAndFkFixes changed the LineItem → Variant foreign key to NoAction because cascading

deletes from `orders` → `line_items` → `variants` would create a multi-hop cascade cycle (PostgreSQL restriction).

Evidence: `git log: commit bd042088`

5.5.3 Identity

The identity schema extends ASP.NET Identity with custom `User`, `Role`, and `UserPasskey` entities. The `ApplicationDbContext` inherits from `IdentityDbContext<User, Role, Guid, ...>` using `Guid` as the key type throughout.

Evidence: `ApplicationDbContext.cs:28`

5.6 PGVECTOR INTEGRATION

5.6.1 Vector Column Configuration

```
// From Vector.Configuration.cs — updated for multi-model support
builder.Entity<VariantImage>()
    .Property(v => v.Embedding)
    .HasColumnType("vector(2048)");  // Max dimension across all
models (ResNet-50 = 2048)
    // Actual dimensions: Fashion-CLIP=512, ResNet-50=2048,
EfficientNet-B0=1280, CLIP-generic=512

builder.Entity<VariantImage>()
    .Property(v => v.ModelName)
    .HasMaxLength(50)
    .HasDefaultValue("fashion-clip");  // Which model generated
this embedding

builder.Entity<VariantImage>()
    .Property(v => v.VectorDim)
    .HasDefaultValue(512);  // Actual dimension for this model's
vector
```

Design decision: A single `vector(2048)` column accommodates all models. Smaller vectors are right-padded with zeros (standard pgvector behavior). The `model_name` and `vector_dim` columns enable per-model filtering and indexing during evaluation.

5.6.2 Similarity Query (Model-Specific)

```
-- Query embeddings from a specific model (e.g., Fashion-CLIP)
SELECT vi.*, v.sku, p.name
FROM catalog.variant_images vi
JOIN catalog.variants v ON vi.variant_id = v.id
JOIN catalog.products p ON v.product_id = p.id
WHERE vi.model_name = 'fashion-clip'
```

```
ORDER BY vi.embedding <=> @query_embedding  -- cosine distance
LIMIT 20;
```

Per-model indexing: During evaluation, separate IVF flat indexes are created per `model_name` to prevent cross-model interference:

```
CREATE INDEX idx_embedding_fashion_clip ON catalog.variant_images
USING ivfflat (embedding vector_cosine_ops)
WHERE model_name = 'fashion-clip';
```

Design decision: Cosine similarity (`<=>` operator in pgvector) is used because Fashion-CLIP embeddings are normalized L2 vectors where cosine distance is equivalent to Euclidean distance and performs well for visual similarity.

Evidence: `Shared/Operational/Persistence/Configurations/Vectors/Vector.Configuration.cs`, `ImageEmbedding.Inference.cs` (returns 512-d vector)

5.7 INDEXING STRATEGY

Table 36.

Table 36: Indexing strategy overview

Index	Purpose	Evidence
`slug` UNIQUE on `products`	Fast lookup by SEO-friendly URL	`CatalogSchema.cs`
`sku` UNIQUE on `variants`	Inventory lookup by SKU	`CatalogSchema.cs`
`number` UNIQUE on `orders`	Order lookup by human-readable number	`OrderingSchema.cs`
Composite `(user_id, status)` on `orders`	My-orders queries	Migration `20260713131410`
Composite `(session_id, status)` on `orders`	Guest cart retrieval	Migration `20260713131410`
`embedding` IVFFlat on `variant_images`	Approximate nearest neighbor search	`Vector.Configuration.cs`

5.8 MIGRATION STRATEGY

EF Core migrations are stored in a **separate assembly** (`Api.Migrations`) to keep the domain and host projects free of migration clutter.

Current migrations:

Table 37.

Table 37: EF Core migration history

Migration	Description
`20260711090657_InitialCreate`	Baseline schema for all 8 modules + Identity — 104 KB (C#), 153 KB (Designer)
`20260712050728_FixPaymentMethodSettingsColumnType`	Changed `payment.payment_method.settings` from `jsonb` to `text`
`20260713131410_OrderingIndexAndFkFixes`	Added composite indexes; changed `LineItem-Variant` FK to `NoAction` — 172 KB (Designer)

Design rationale: Large initial migrations are expected when bootstrapping a modular schema. The fix migration demonstrates the project's iterative approach: schema changes are versioned and reversible.

Evidence: `service/Api/src/Migrations/Migrations/`

5.9 EVIDENCE

- `service/Api/src/Shared/Operational/Persistence/Data/AppDbContext.cs:1-113` — DbContext with pgvector extension, schema configuration discovery
- `service/Api/src/Shared/Operational/Persistence/Configurations/Vectors/Vector.Configuration.cs` — pgvector column setup
- `service/Api/src/Module/Catalog/Persistence/CatalogSchema.cs` — schema constants
- `service/Api/src/Migrations/Migrations/20260711090657_InitialCreate.cs` — baseline migration
- `service/Api/src/Migrations/Migrations/20260713131410_OrderingIndexAndFkFixes.cs` — index optimization
- `infra/Aspire/src/ReSys.AppHost/AppHost.cs:11-12` — PostgreSQL 17 + pgvector image selection
- `service/Api/src/Module/Catalog/Domain/Products/Variants/Images/VariantImage.cs` — entity with `Embedding` property

6 CHAPTER 5: API DESIGN

6.1 API STYLE

Decision: Minimal API endpoints using **Carter** (`ICarterModule`) with explicit route mapping, rather than traditional ASP.NET MVC controllers.

Justification:

Table 38.

Table 38: Carter Minimal API vs. MVC Controllers comparison

Aspect	Carter Minimal API	MVC Controllers
Boilerplate	One file per endpoint, no `[Route]` attributes	Controller class + action method + attribute routing
Discoverability	`AddEndpoints()` scans assemblies for `ICarterModule`	Reflection-based controller discovery
Cohesion with Vertical Slice	Endpoint class lives in the same folder as handler	Controllers typically in separate `Controllers/` folder
Performance	Slightly faster routing (delegates, not action descriptors)	Negligible difference at thesis scale

The endpoint convention uses a `static partial class` with a nested `Endpoint` class implementing `ICarterModule`:

```
public static partial class CreateProduct
{
    public sealed class Endpoint : ICarterModule
    {
        public void AddRoutes(IEndpointRouteBuilder app)
        {
app.MapPost(CatalogFeature.Admin.Products.Create.Route, ...)
             .HasPermission(CatalogPermission.AdminProductsCreate)
              .WithTags(CatalogFeature.Admin.Products.Create.Tags)
           .WithSummary(CatalogFeature.Admin.Products.Create.Summary)
                .Produces<Response>(StatusCodes.Status201Created)
                .ProducesValidationProblem();
        }
    }
}
```

Evidence: `Module/Catalog/Features/Admin/Products/Create/CreateProduct.Endpoint.cs:14-32`

6.2 AUTHENTICATION AND AUTHORIZATION

6.2.1 Authentication Model

- **Primary**: JWT Bearer tokens (`Authorization: Bearer <access_token>`)
  - Access token expiry: 15 minutes
  - Refresh token expiry: 7 days
  - Algorithm: HS256 (HMAC-SHA256)
  - Secret stored in `dotnet user-secrets` for dev; env var for production
- **Secondary**: Google OAuth 2.0 for external login
  - Returns JWT pair same as password login
- **Guest**: Cookie-based guest session (`Guest` cookie, 30-day expiry, HttpOnly, Secure, SameSite=Lax)

Evidence: `appsettings.json:30-57`, `Shared/Security/Authentication/Tokens/Tokens.Extensions.cs:33-88`

6.2.2 Authorization Model

Permission-based authorization using a custom `IAuthorizationPolicyProvider`:

1. Each module declares `*FeatureMetadata` types containing route, summary, tags, and **permission descriptor**
2. Endpoints call `.HasPermission(...)` which registers a policy name dynamically

3. `PermissionPolicyProvider` maps policy name → `PermissionRequirement`
4. `PermissionContext` enumerates all allowed permission values as a registry
5. ASP.NET Identity claims store the user's permissions; the handler checks claims against the requirement

Permission format: `{Domain}:{Category}:{Action}` (e.g., `catalog:products:create`, `ordering:orders:cancel`)

Evidence: `Shared/Security/Authorization/Policies/Permission.PolicyProvider.cs:1-31`, `Shared/Security/Authorization/Registry/PermissionContext.cs:1-60`, `Shared/Security/Authorization/Attributes/HasPermission.Attribute.Extension.cs`

6.3 REQUEST / RESPONSE MODELS

6.3.1 Unified Error Envelope

All failures return a consistent JSON structure:

```
{
  "isSuccess": false,
  "statusCode": 400,
  "errors": [
    { "code": "Slug.Unique", "message": "A product with this slug
already exists." }
  ],
  "message": "One or more validation errors occurred.",
  "metadata": null
}
```

Design rationale: A unified envelope makes client error handling predictable. The `code` field allows clients to implement i18n by mapping codes to localized messages.

Evidence: `Shared/Application/Models/Results/Result.Method.cs:65-186`, `Models/Errors/Error.cs`

6.3.2 Sample Endpoint: Create Product (Admin)

Request:

```
POST /api/catalog/admin/products
Content-Type: application/json
Authorization: Bearer <token>
X-CSRF-TOKEN: <anti-forgery-token>

{
  "name": "Summer Floral Dress",
  "description": "Light cotton dress with floral print",
  "slug": "summer-floral-dress",
  "status": "Draft",
```

```
  "metaTitle": "Summer Floral Dress | ReSys",
  "metaDescription": "...",
  "styleCode": "FLR-2026-001",
  "seasonName": "Summer 2026",
  "materialComposition": "100% Cotton",
  "careInstructions": "Machine wash cold",
  "fitNotes": "True to size",
  "department": "Women",
  "genderTarget": "Female",
  "availableOn": "2026-04-01T00:00:00Z"
}
```

Response (201 Created):

```
{
  "isSuccess": true,
  "statusCode": 201,
  "data": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "name": "Summer Floral Dress",
    "slug": "summer-floral-dress",
    "masterVariantId": "b2c3d4e5-f6a7-8901-bcde-f23456789012"
  }
}
```

Evidence: `ApiTests/Catalog/Admin/products.http`, `CreateProduct.Response.cs`

6.3.3 Sample Endpoint: Search by Image (Storefront)

Request:

```
POST /api/catalog/storefront/search-by-image
Content-Type: multipart/form-data

image: <binary image data>
```

Response (200 OK):

```
{
  "isSuccess": true,
  "statusCode": 200,
  "data": [
    {
      "productId": "...",
      "variantId": "...",
      "name": "Similar Floral Dress",
      "similarityScore": 0.92,
      "imageUrl": "/uploads/..."
    }
  ]
}
```

Evidence: `ApiTests/Catalog/Storefront/search-by-image.http`

6.3.4 Sample Endpoint: Checkout (Storefront)

Request:

```
POST /api/ordering/storefront/cart/checkout
Content-Type: application/json
Authorization: Bearer <token>

{
  "billAddressId": "...",
  "shipAddressId": "...",
  "shippingMethodId": "...",
  "paymentMethodId": "..."
}
```

Response (201 Created):

```
{
  "isSuccess": true,
  "statusCode": 201,
  "data": {
    "orderId": "...",
    "orderNumber": "ORD-2026-00042",
    "total": 129.99,
    "currency": "USD",
    "checkoutState": "Payment",
    "clientSecret": "pi_xxx_secret_yyy"
  }
}
```

Evidence: `ApiTests/Ordering/Cart.http`, `CreateOrderFromCart.cs`

6.4 OPENAPI AND DOCUMENTATION

The API generates OpenAPI 3.0 spec automatically via `Microsoft.AspNetCore.OpenApi`. Scalar UI serves interactive documentation at `/scalar/v1`.

FluentValidation auto-registration: All validators are discovered at startup and registered into the MediatR pipeline. Validation errors are automatically mapped to the error envelope.

Evidence: `Shared/Governance/Governance.Extension.cs:1-57`, `Directory.Packages.props:53,55`

6.5 HTTP TEST ARTIFACTS AS LIVING DOCUMENTATION

The `ApiTests/` directory contains 49 `.http` files that serve as:

- Executable API documentation (REST Client / JetBrains HTTP Client)
- Manual QA scripts
- Thesis evidence of endpoint coverage

Evidence: `ApiTests/README.md:1-30`, `ApiTests/run-all.http`

6.6 EVIDENCE

- `service/Api/src/Shared/Application/Endpoints/Endpoint.Extension.cs:1-63` — Carter scanning and `ToResult()` mapping
- `service/Api/src/Shared/Governance/OpenApi/OpenApi.Extension.cs:54-62` — Scalar UI registration
- `service/Api/src/Shared/Security/Authorization/Attributes/HasPermission.Attribute.Extension.cs` — permission endpoint convention
- `service/Api/src/Module/Catalog/Features/Admin/Products/Create/CreateProduct.Endpoint.cs` — endpoint definition
- `service/Api/src/Module/Catalog/Features/Admin/Products/Create/CreateProduct.Request.cs` — request DTO
- `service/Api/src/Module/Catalog/Features/Admin/Products/Create/CreateProduct.Response.cs` — response DTO
- `ApiTests/` — manual HTTP test artifacts

7 CHAPTER 6: DETAILED DESIGN

7.1 COMPONENT-LEVEL DESIGN

7.1.1 MediatR Pipeline (Decorator Chain)

The pipeline is the backbone of the system's cross-cutting concern handling. It is registered in this order (outermost first):

```
LoggingBehavior<TRequest, TResponse>
  → ValidationBehavior<TRequest, TResponse>
    → ExceptionMappingBehavior<TRequest, TResponse>
      → Handler
```

Behavior responsibilities:

Table 39.

Table 39: MediatR pipeline behavior responsibilities

Behavior	Order	Responsibility	Short-circuit?
`LoggingBehavior`	1 (outer)	Log request entry/exit with CorrelationId, elapsed ms	No
`ValidationBehavior`	2	Run FluentValidation; return `Result.Validation` on failure	Yes
`ExceptionMappingBehavior`	3	Catch unhandled exceptions; return `Result.Unexpected`	No (catch-all)

Design rationale: Validation is placed **inside** logging so that validation failures are still logged. Exception mapping is innermost to catch anything that leaks past validation.

Evidence: `Shared/Application/Mediators/Mediator.Extension.cs:46-50`

7.1.2 Create Product Flow

The Create Product flow demonstrates the vertical slice pattern in action. The following describes the interaction between system components:

1. The **Client** sends a `POST /api/catalog/admin/products` request to the **Carter Endpoint**
2. The **Carter Endpoint** dispatches a `CreateProduct` command via `ISender.Send()` to the **MediatR Pipeline**
3. The pipeline runs the **ValidationBehavior** which invokes the FluentValidation validator; on failure, returns early
4. On success, the **CreateProduct Handler** is invoked
5. The handler queries **Application DbContext** to verify slug uniqueness
6. The handler invokes the Product factory method to create the domain entity
7. The handler calls `SaveChanges()` to persist the Product
8. The handler dispatches an `AddVariant` nested command to create the master variant
9. The handler sets `MasterVariantId` on the Product and saves again
10. The handler maps the result to `CreateProduct.Response` via Mapster and returns 201 Created

Evidence: `CreateProduct.cs:36-78`, `CreateProduct.Endpoint.cs:14-32`

7.1.3 Checkout Flow

The Checkout flow orchestrates order creation, inventory reservation, and payment initiation:

1. The **Client** sends a `POST /api/ordering/storefront/cart/checkout` request to the **Carter Endpoint**
2. The **Carter Endpoint** dispatches a `CreateOrderFromCart` command via the MediatR pipeline
3. The **CreateOrderFromCart Handler** validates the cart is not empty and validates stock availability
4. The handler begins a `RepeatableRead` transaction on **Application DbContext**
5. The handler generates an order number, creates the Order and LineItems entities
6. The handler calls `SaveChanges()` to persist the order within the transaction
7. The handler invokes the **Payment Gateway** to create a PaymentIntent via the **Stripe API**
8. On success, the handler commits the transaction
9. The handler maps the result to the checkout response via Mapster and returns 201 Created

Design decision: Order number generation and all related inserts happen inside a `RepeatableRead` transaction to prevent phantom reads on inventory during high-concurrency checkout scenarios.

Evidence: `git log: commit 887a77c7`, `CreateOrderFromCart.cs`

7.1.4 Image Search Flow (CBIR — Model-Agnostic)

The Content-Based Image Retrieval flow connects the frontend, backend, Python sidecar, and database:

1. The **Client** uploads an image to the **Store SPA**
2. The **Store SPA** sends a `POST /search-by-image` multipart request to the **Catalog Backend (Search Handler)**
3. The **Catalog Backend** forwards the image bytes and `model=fashion-clip` parameter to the **Embedding Sidecar**
4. The **Embedding Sidecar** lazily initializes the requested model (Fashion-CLIP) and calls `encode_image()`
5. The sidecar returns a 512-dimensional embedding vector and model name to the **Catalog Backend**
6. The **Catalog Backend** executes a vector similarity query against **PostgreSQL + pgvector** using the cosine distance operator (`<=>`), filtering by model name and ordering by similarity
7. The top-K results are returned to the **Store SPA** and displayed to the client

Model abstraction in sidecar: The sidecar uses a **Strategy pattern** with a `BaseEmbeddingModel` abstract class. Each model (Fashion-CLIP, ResNet-50, EfficientNet-B0, CLIP-generic) is a concrete implementation loaded at runtime

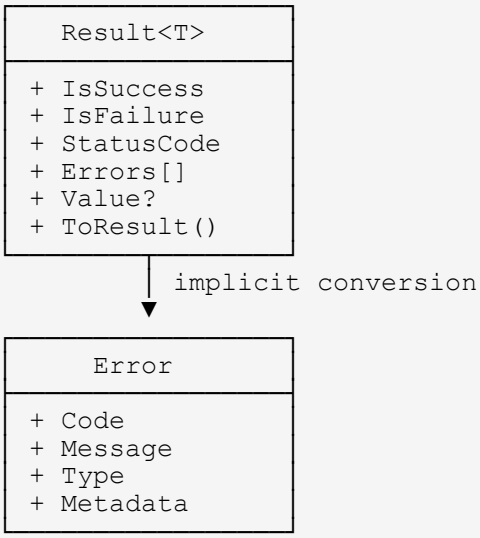
based on the `EMBEDDING_MODEL` environment variable. This enables swapping models without code changes.

Evidence: `ImageEmbedding.Inference.cs:21-36`, `ApiTests/Catalog/Storefront/search-by-image.http`

7.2 CLASS DIAGRAMS

7.2.1 Result Type Hierarchy

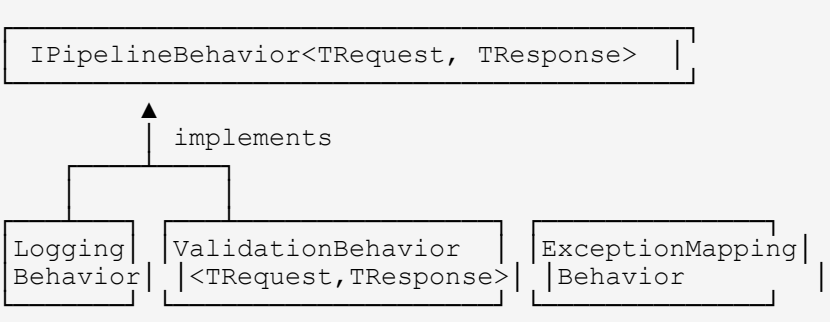
The `Result<T>` type is the universal return type for all domain operations. It encapsulates either a success value or a collection of errors:



Evidence: `Shared/Application/Models/Results/Result.cs`, `Shared/Application/Models/Errors/Error.cs`

7.2.2 Pipeline Behavior Hierarchy

The MediatR pipeline behaviors all implement `IPipelineBehavior<TRequest, TResponse>`:



Evidence: `Shared/Application/Mediators/Behaviours/Logging/Logging.Behaviours.cs`, `Validation/Validation.Behavior.cs`, `Exceptions/Exception.Behavior.cs`

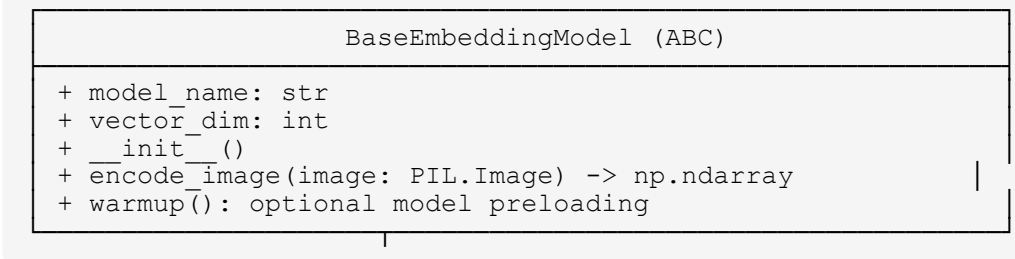
7.3 ML SERVICE WORKFLOW

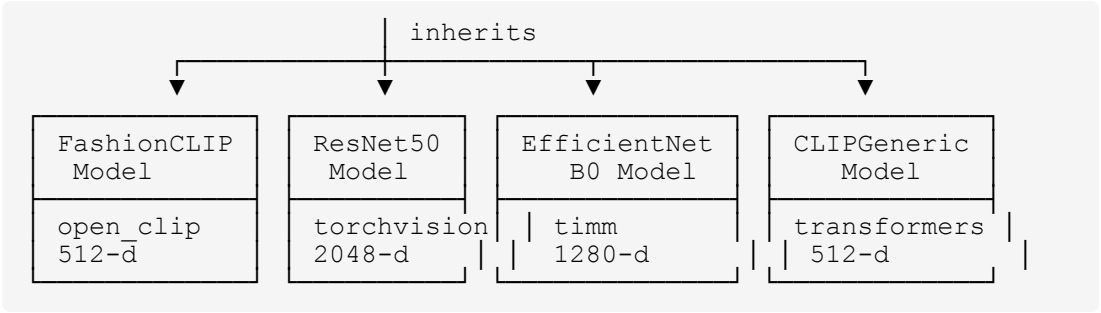
The Python sidecar is a stateless FastAPI application that performs one primary function: image embedding generation. It is architected for **pluggability**: multiple pretrained models can be swapped at runtime via environment variable, supporting the comparative evaluation in Chapter 11.

7.3.1 Architecture

```
service/Embedding/
├── src/
│   ├── main.py              # FastAPI app, CORS, exception
handlers
│   ├── routers/
│   │   ├── embedding_router.py   # POST /embeddings (accepts
model param)
│   │   ├── health_router.py      # GET /health
│   │   └── model_router.py       # GET /models (lists available
models)
│   ├── services/
│   │   └── embedding_service.py  # Delegates to
BaseEmbeddingModel
│   ├── models/
│   │   ├── base_model.py         # Abstract base: encode_image()
→ np.ndarray
│   │   ├── clip_model.py         # Fashion-CLIP (open_clip) —
512-d
│   │   ├── resnet_model.py       # ResNet-50 (torchvision) —
2048-d
│   │   ├── efficientnet_model.py # EfficientNet-B0 (timm) —
1280-d
│   │   └── clip_generic_model.py # CLIP-generic (transformers) —
512-d
│   └── utils/
│       └── image_preprocessing.py # Resize, normalize, tensor
conversion
└── tests/
    ├── unit/
    ├── integration/
    └── e2e/
```

7.3.2 Embedding Model Class Hierarchy (Strategy Pattern)





Model selection at runtime:

```
# From embedding_service.py
MODEL_REGISTRY = {
    "fashion-clip": FashionCLIPModel,
    "resnet50": ResNet50Model,
    "efficientnet_b0": EfficientNetB0Model,
    "clip": CLIPGenericModel,
}

def get_model() -> BaseEmbeddingModel:
    model_name = os.getenv("EMBEDDING_MODEL", "fashion-clip")
    return MODEL_REGISTRY[model_name]()
```

7.3.3 Embedding Generation Flow

1. **Input**: Multipart image upload (JPEG/PNG/WebP) + optional `model` query parameter
2. **Model resolution**: `embedding_service.py` reads `model` param or `EMBEDDING_MODEL` env var → instantiates correct `BaseEmbeddingModel` subclass
3. **Preprocessing**: Model-specific preprocessing (e.g., Fashion-CLIP: 224×224; ResNet-50: 224×224; EfficientNet-B0: 224×224 with B0-specific normalization)
4. **Inference**: `model.encode_image(image)` → `np.ndarray` (variable dimension: 512, 2048, or 1280)
5. **Output**: JSON `{"embedding": [0.0123, ...], "model_name": "fashion-clip", "vector_dim": 512}`

Evidence: `service/Embedding/src/main.py:1-29`, `service/Embedding/pyproject.toml:15-16`

7.4 EVIDENCE

- `service/Api/src/Shared/Application/Mediators/Mediator.Extension.cs:1-79` — pipeline registration
- `service/Api/src/Shared/Application/Mediators/Behaviours/Validation/Validation.Behavior.cs:1-67` — validation behavior

- `service/Api/src/Shared/Application/Mediators/Behaviours/Exceptions/Exception.Behavior.cs:1-42` — exception mapping
- `service/Api/src/Module/Catalog/Features/Admin/Products/Create/CreateProduct.cs:36-78` — handler logic
- `service/Api/src/Module/Ordering/Features/Storefront/Cart/Checkout/CreateOrderFromCart.cs` — checkout handler
- `service/Api/src/Module/Catalog/Features/Admin/Products/Variants/Images/Embeddings/Shared/Clients/ImageEmbedding.Inference.cs` — embedding client
- `service/Embedding/src/main.py:1-29` — Python service entry
- `service/Embedding/src/models/base_model.py` — embedding model abstraction
- `service/Embedding/src/models/clip_model.py` — Fashion-CLIP implementation
- `service/Embedding/src/models/resnet_model.py` — ResNet-50 implementation
- `service/Embedding/src/models/efficientnet_model.py` — EfficientNet-B0 implementation
- `service/Embedding/src/models/clip_generic_model.py` — CLIP-generic implementation
- `service/Api/src/Shared/Application/Models/Results/Result.cs:1-43` — Result type

8 CHAPTER 7: SECURITY DESIGN

8.1 THREAT MODEL AND DESIGN PRINCIPLES

Security design follows defense-in-depth with multiple independent controls at each layer:

Table 40.

Table 40: Defense-in-depth security controls by layer

Layer	Controls
Network	CORS allow-list, rate limiting, security headers
Authentication	JWT with short expiry, refresh-token rotation, guest sessions
Authorization	Permission-based claims, dynamic policy provider
Transport	HTTPS only (Aspire local dev uses HTTP for simplicity)
Input	FluentValidation, anti-forgery tokens, file upload guards
Data	EF Core parameterization (SQL injection prevention), no secrets in config
Observability	Sensitive header redaction, correlation IDs without PII

Evidence: `appsettings.json:30-155`, `Shared/Security/`

8.1.1 Threat Model Approach

Decision: The thesis documents a **defense-in-depth controls table** (§8.1) rather than an exhaustive per-element STRIDE analysis. The rationale is that the security contribution of this thesis is architectural — explicit error handling, permission-based authorization, and JWT token design — rather than a dedicated security research contribution. A full STRIDE threat model across all 18 entity types would add 5 pages without advancing the primary thesis argument.

If required by examiner: A structured STRIDE analysis for the three most critical elements (Order, PaymentIntent, JWT Token) is prepared and can be appended as a supplementary document. It maps Spoofing, Tampering, Repudiation, Info Disclosure, DoS, and Elevation threats to the existing mitigations and identifies one gap (synchronous Stripe webhook processing, already tracked in `CONCERNS.md`).

Evidence: `Shared/Security/Authentication/Tokens/Tokens.Extensions.cs:33-88`, `Module/Payment/Features/Storefront/Payment/Webhooks/StripeWebhook.cs:32-36`, `Shared/Security/Authorization/Policies/Permission.PolicyProvider.cs:1-31`, `CONCERNS.md:§2`

8.2 AUTHENTICATION DESIGN

8.2.1 JWT Token Design

Access Token:
- Type: JWT signed with HS256
- Expiry: 15 minutes (`appsettings.json:35`)
- Claims: `sub` (user id), `email`, `jti` (token id), `iat`, `exp`

- Issuer/Audience: `ReSys.Shop`

Refresh Token:
- Stored in database (`UserToken` table with `TokenType = Refresh`)
- Expiry: 7 days
- **Rotation**: New refresh token issued on every access-token renewal; old token invalidated
- **Reuse detection**: If a previously-used refresh token is presented again, all tokens for that user are revoked (theft detection)
- **Blacklist**: Revoked tokens stored in Redis for fast lookup

Design rationale: Short access-token lifetime limits the window of compromise. Refresh-token rotation + reuse detection prevents replay attacks if a token is stolen. The blacklist in Redis ensures O(1) revocation checks.

Evidence: `appsettings.json:30-43`, `Shared/Security/Authentication/Tokens/Services/Refresh/`

8.2.2 Dev Secret Handling

A historical vulnerability (commit `770b6a06`) allowed hardcoded dev JWT secrets to work in non-Development environments. This was mitigated by:

1. Moving dev secrets to `dotnet user-secrets` (id `resys.shop.api`)
2. `JwtSettingsValidator` rejects known-dev literal secrets in non-Development environments
3. `setup-dev-secrets.sh` bootstraps dev secrets safely

Evidence: `AGENTS.md:66`, `appsettings.Development.json:1-2`, `Tokens.Extensions.cs:38-43`

8.2.3 External Identity Providers

- **Google OAuth**: Implemented using `Google.Apis.Auth`
- **Facebook / Microsoft**: Config placeholders exist but are disabled (`Enabled=false`); not implemented

Evidence: `appsettings.json:45-57`, `Shared/Security/Authentication/External/ExternalLogin.Extensions.cs`

8.3 AUTHORIZATION DESIGN

8.3.1 Permission-Based Authorization

The system uses a custom authorization model rather than simple role-based access control (RBAC):

1. **Permission descriptors** are defined in `PermissionContext` as:
   - `Domain` (e.g., `catalog`, `ordering`)
   - `Category` (e.g., `products`, `orders`)
   - `Action` (e.g., `create`, `read`, `update`, `delete`, `cancel`)
2. **Feature metadata** types (e.g., `CatalogFeature.Admin.Products.Create`) bundle route, tags, summary, and permission descriptor.
3. **Endpoint registration** calls `.HasPermission(CatalogPermission.AdminProductsCreate)` which creates a policy string like `catalog:products:create`.
4. **Policy provider** (`PermissionPolicyProvider`) resolves the string to a `PermissionRequirement`.
5. **Handler** checks if the user's claims contain the required permission.

Design rationale: RBAC is too coarse for e-commerce (an admin may edit products but not process refunds). Permission-based authorization gives granular control while keeping the policy provider centralized.

Evidence: `Shared/Security/Authorization/Registry/PermissionContext.cs:1-60`, `Permission.PolicyProvider.cs:1-31`

8.3.2 Admin vs Storefront Surface Segregation

Features are physically separated into `Features/Admin/` and `Features/Storefront/` folders. Endpoints in `Admin/` typically require `catalog:products:create`-style permissions, while `Storefront/` endpoints rely on authentication state (or guest session) without explicit permission checks for browsing.

Evidence: `Module/*/Features/Admin/` and `Module/*/Features/Storefront/` directory structures

8.4 INPUT VALIDATION AND ANTI-FORGERY

8.4.1 FluentValidation Pipeline

Every request DTO has a corresponding `Validator.cs` class. The `ValidationBehavior` runs all validators before the handler executes. Validation failures short-circuit the pipeline and return `Result.Validation` with field-level error codes.

Evidence: `Validation.Behavior.cs:1-67`, `CreateProduct.Validator.cs`

8.4.2 Anti-Forgery Tokens

Enabled by default (`appsettings.json:69-78`):
- Header name: `X-CSRF-TOKEN`
- Cookie: `.AspNetCore.Antiforgery`
- SameSite: `Strict`
- Secure: `Always` (requires HTTPS)
- Required: `true` (all mutating endpoints must include token)

Design rationale: The Storefront SPA uses JWT Bearer auth (stateless), so CSRF protection is less critical for API endpoints. The anti-forgery system primarily protects cookie-based flows (admin login, guest sessions).

Evidence: `appsettings.json:69-78`, `Shared/Security/AntiForgery/AntiForgery.Extensions.cs`

8.4.3 File Upload Security

Multi-layered defense for uploaded files:

Table 41.

Table 41: File upload security controls

Layer	Check	Implementation
1. Magic bytes	File header matches extension	`IStorageSecurityEnforcer`
2. Extension allowlist	Only `.jpg`, `.png`, `.webp`, `.pdf`, etc.	`appsettings.json:135-138`
3. Extension blocklist	`.exe`, `.bat`, `.ps1`, `.jar` blocked	`appsettings.json:139-142`
4. Size limit	Max 10 MB	`appsettings.json:134`
5. Anti-forgery guard	Rate-limit consecutive failures	`appsettings.json:146-149`
6. Malware scan	ClamAV TCP scan (opt-in, disabled by default)	`appsettings.json:150-155`

Evidence: `Shared/Operational/Storages/Storage.Extensions.cs:35-74`, `appsettings.json:129-155`

8.5 RATE LIMITING

Named policies protect specific endpoint categories:

Table 42.

Table 42: Rate limit policies

Policy	Permit Limit	Window	Protected Endpoints
`default`	100	60s	All unclassified
`auth`	5	60s	Login, token refresh
`register`	3	3600s	Registration
`forgot-password`	3	3600s	Password reset
`payment`	30	60s	Payment intent creation

Design rationale: Rate limiting prevents brute-force attacks on auth endpoints and reduces fraud risk on payment endpoints. The `payment` policy is higher (30/min) than auth because legitimate checkout flows may involve multiple payment attempts.

Evidence: `appsettings.json:79-86`, `Shared/Security/RateLimiting/RateLimit.Extensions.cs`

8.6 SECURITY HEADERS

A custom middleware injects security headers on every response:

Table 43.

Table 43: Security response headers

Header	Value	Purpose
`X-Content-Type-Options`	`nosniff`	Prevent MIME-type sniffing
`X-Frame-Options`	`DENY`	Clickjacking protection
`Content-Security-Policy`	`default-src 'self'`	XSS mitigation
`Strict-Transport-Security`	`max-age=31536000`	HSTS
`Referrer-Policy`	`strict-origin-when-cross-origin`	Privacy

Evidence: `Shared/Security/Headers/SecurityHeadersMiddleware.cs`

8.7 SECRETS MANAGEMENT

Table 44.

Table 44: Secrets management by environment

Environment	Secret Store	Evidence
Development	`dotnet user-secrets` (id `resys.shop.api`)	`Api.csproj:7`, `setup-dev-secrets.sh`
Testing	Hardcoded in `ApiFactory.cs` (self-contained)	`ApiFactory.cs:53,81-83`
Production	Environment variables (`__` delimiter)	`.env.template:5-33`

No secrets are committed to the repository. All `appsettings.json` secret fields are empty strings (`""`).

Evidence: `appsettings.json` (empty secrets), `.env.template`, `AGENTS.md:66`

8.8 EVIDENCE

- `service/Api/src/Api/appsettings.json:30-155` — security configuration
- `service/Api/src/Shared/Security/Authentication/Tokens/Tokens.Extensions.cs:33-88` — JWT setup
- `service/Api/src/Shared/Security/Authorization/Registry/PermissionContext.cs:1-60` — permission registry
- `service/Api/src/Shared/Security/RateLimiting/RateLimit.Extensions.cs` — rate limit policies
- `service/Api/src/Shared/Security/AntiForgery/AntiForgery.Extensions.cs` — anti-forgery setup
- `service/Api/src/Shared/Security/Headers/SecurityHeadersMiddleware.cs` — response headers
- `service/Api/src/Shared/Operational/Storages/Storage.Extensions.cs:35-74` — upload security
- `service/Api/src/Api/.env.template:1-33` — required environment variables

9 CHAPTER 8: DEPLOYMENT DESIGN

9.1 DEPLOYMENT ARCHITECTURE

9.1.1 Local Development (Aspire Orchestration)

Aspire provides a single-command local development environment. The `AppHost` project orchestrates all services:

Table 45.

Table 45: Aspire-orchestrated local development services

Service	Port	Runtime
PostgreSQL pgvector:17	5432	Container (Docker)
Redis 7-alpine	6379	Container (Docker)
API	5035	.NET 10 (Kestrel)
Embedding sidecar	8000	Python FastAPI
Store SPA (Vite dev)	5174	Node.js
Admin SPA (Vite dev)	5173	Node.js

All containers and processes are managed by the Aspire AppHost. The frontends are dev-served by Vite and proxy API requests to `localhost:5035`.

Evidence: `infra/Aspire/src/ReSys.AppHost/AppHost.cs:9-49`

9.1.2 Service Defaults

All Aspire-managed services share `ReSys.ServiceDefaults` which registers:

- OpenTelemetry (traces, metrics, logs)
- Health checks (`/health`, `/alive`)
- Service discovery
- HTTP client resilience (Polly pipeline)

Evidence: `infra/Aspire/src/ReSys.ServiceDefaults/Extensions.cs:19-132`

9.1.3 Production Deployment (Conceptual)

Since Dockerfiles and CI/CD are explicitly out of scope (deferred), the production deployment design is conceptual. The architecture assumes:

Table 46.

Table 46: Production deployment targets

Component	Deployment Target
API container	Single .NET 10 container (modular monolith)
Store SPA	Static bundle on CDN or object storage
Admin SPA	Static bundle on CDN or object storage
Embedding sidecar	Separate Python container
PostgreSQL 17	Managed cloud database
Redis 7	Managed cloud cache/job queue
Load balancer	YARP reverse proxy (deferred)
External services	Stripe API, SendGrid API, S3 bucket

Design rationale: A modular monolith is naturally containerizable as a single API container. The frontends are static SPA bundles served from a CDN or object storage. The embedding sidecar is a separate container because Python/.NET runtimes don't share a process. Redis and PostgreSQL are best managed by cloud providers in production.

9.1.4 Deployment Diagram (Conceptual)

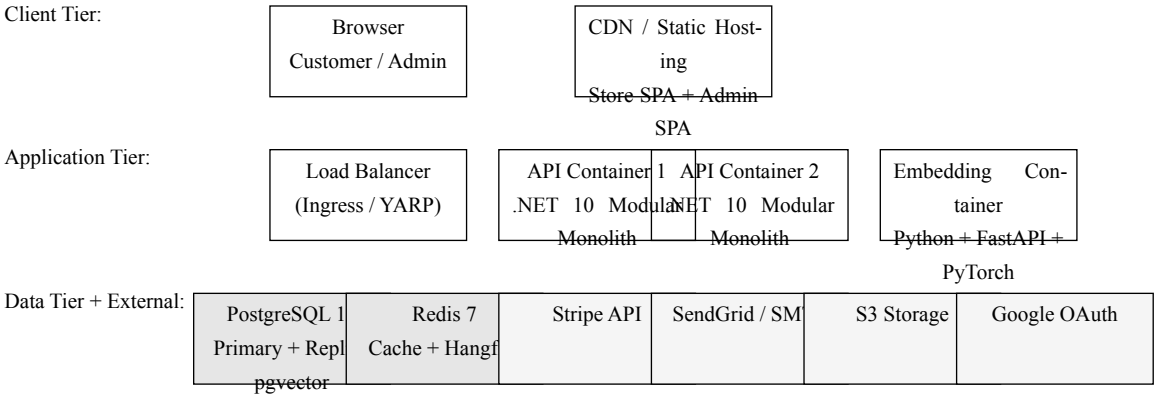


Figure 1.

Figure 1: Conceptual Production Deployment Diagram

9.2 CONFIGURATION PER ENVIRONMENT

Table 47.

Table 47: Configuration source precedence by environment

Source	Development	Testing	Production
`appsettings.json`	Base	Base	Base
`appsettings.Development.json`	Override	Ignored	Ignored
`appsettings.Testing.json`	Ignored	Override	Ignored
`dotnet user-secrets`	Secrets	Ignored	Ignored
Environment variables	Optional	Injected by test factory	Primary secret source

Evidence: `service/Api/src/Api/appsettings.json`, `appsettings.Development.json`, `appsettings.Testing.json`, `.env.template`

9.3 HEALTH CHECKS

Three health checks are registered:

Table 48.

Table 48: Health check endpoints and purposes

Check	Endpoint	Purpose
`self`	`/alive`	Liveness — process is running
`npgsql`	`/health`	Readiness — database connectivity
`redis`	`/health`	Readiness — cache connectivity
`database_initialization`	`/health`	Readiness — migrations have run

Note: `MapDefaultEndpoints` only exposes these in non-production environments (`Extensions.cs:114-131`). In production, health checks should be exposed on a separate port or via a sidecar.

Evidence: `infra/Aspire/src/ReSys.ServiceDefaults/Extensions.cs:114-131`

9.4 EVIDENCE

- `infra/Aspire/src/ReSys.AppHost/AppHost.cs:1-49` — Aspire orchestration
- `infra/Aspire/src/ReSys.ServiceDefaults/Extensions.cs:1-132` — service defaults
- `service/Api/src/Api/appsettings.json:1-237` — environment config hierarchy
- `service/Api/src/Api/.env.template:1-33` — environment variable documentation

10 CHAPTER 9: TESTING STRATEGY

10.1 TESTING PYRAMID

The project implements a four-layer testing strategy:

```
      ┌──────────────┐
      │   Manual     │  ← ApiTests/ (49 .http files)
      │   API Tests  │     REST Client / JetBrains HTTP Client
      ├──────────────┤
      │  Integration │  ← Api.Tests (Testcontainers +
WebApplicationFactory)
      │   (Backend)  │     Docker required; tests real DB + HTTP
pipeline
      ├──────────────┤
      │    Unit      │  ← Module.UnitTests + Shared.UnitTests
      │   (Backend)  │     InMemory EF; Moq for dependencies; no
Docker
      ├──────────────┤
      │    Unit      │  ← Vitest (Admin + Storefront SPAs)
      │  (Frontend)  │     jsdom + @vue/test-utils
      └──────────────┘
```

10.2 BACKEND TESTING

10.2.1 Unit Tests (Module.UnitTests + Shared.UnitTests)

Scope: Handlers, validators, domain methods, and utility classes in isolation.

Technique:

- `ApplicationDbContext` uses `UseInMemoryDatabase(Guid.NewGuid().ToString())` for each test class
- `Mock<ISender>` for nested command dispatch
- `Mock<ILogger<T>>` for logger verification
- `Mock<ICurrentUser>` for user context simulation
- `ApplicationDbContext.AdditionalConfigurationsAssemblies` must be set **before** first use to load module entity configs

Sample test pattern (Create Product):

```
[Trait("Category", "Unit")]
[Trait("Module", "Catalog")]
[Trait("Feature", "ProductCreate")]
public sealed class CreateProductTests
{
    [Fact]
    public async Task
Handle_ValidRequest_CreatesProductWithMasterVariant()
    {
        // Arrange
        var db = new ApplicationDbContext(new
DbContextOptionsBuilder<ApplicationDbContext>()
         .UseInMemoryDatabase(Guid.NewGuid().ToString()).Options);
        db.AdditionalConfigurationsAssemblies =
[typeof(Product).Assembly];
        var sender = new Mock<ISender>();
        var logger = new
Mock<ILogger<CreateProduct.CommandHandler>>();
        var currentUser = new Mock<ICurrentUser>();
        var handler = new CreateProduct.CommandHandler(db,
sender.Object, logger.Object, currentUser.Object);

        // Act
        var result = await handler.Handle(new
CreateProduct.Command(request),
TestContext.Current.CancellationToken);

        // Assert
        result.IsSuccess.Should().BeTrue();
result.StatusCode.Should().Be(StatusCodes.Status201Created);
    }
}
```

Evidence: `service/Api/tests/Module.UnitTests/Catalog/Features/Admin/Products/Create/CreateProduct.Tests.cs:1-92`

10.2.2 Integration Tests (Api.Tests)

Scope: Full HTTP pipeline including middleware, auth, validation, database, and external service fakes.

Technique:

- `ApiFactory : WebApplicationFactory<Program>` boots the real host with in-memory config overrides
- `Testcontainers.PostgreSql` spins up a real PostgreSQL container
- `Respawn` checkpoints the database state between tests for fast reset
- `TestCurrentUser` uses `AsyncLocal<Guid?>` to simulate different users within a test
- All external integrations disabled: `Caching:* = false`, `BackgroundJobs:Enabled = false`, `Storage:Enabled = false`, `Storage:MalwareScanner:Enabled = false`
- `IHostedService` implementations removed to prevent background DB initializer from running

Evidence: service/Api/tests/Api.Tests/Infrastructure/ApiFactory.cs:1-189

10.2.3 Test Commands

```
# Fast feedback (no Docker)
dotnet test service/Api/tests/Module.UnitTests
dotnet test service/Api/tests/Shared.UnitTests

# Full pipeline (requires Docker)
dotnet test service/Api/tests/Api.Tests

# With coverage
dotnet test /p:CollectCoverage=true

# Filter by module
dotnet test --filter "FullyQualifiedName~Location"
```

Evidence: Directory.Build.props:95-98, AGENTS.md:42-51

10.3 FRONTEND TESTING

10.3.1 Admin SPA

- **Runner**: Vitest 4.1.9 with jsdom 29.1.1
- **Pattern**: Colocated __tests__/ or tests/ directories
- **Utilities**: @vue/test-utils for component mounting, createPinia() for state
- **Coverage**: @vitest/coverage-v8 (opt-in)

Evidence: app/Admin/vitest.config.ts:1-14, app/Admin/src/features/auth/_tests/auth.service.spec.ts

10.3.2 Store SPA

- **Runner**: Vitest 4.1.9 with jsdom
- **Pattern**: __tests__/*.spec.ts
- **Example**: App.spec.ts mounts the root App with Pinia + router + Nuxt UI plugins

Evidence: app/Store/vitest.config.ts:1-13, app/Store/src/__tests__/App.spec.ts:1-28

10.3.3 Frontend Commands

```
cd app/Admin && pnpm run test:unit   # vitest run
cd app/Admin && pnpm run lint        # oxlint + eslint
cd app/Store && pnpm run test:unit
```

10.4 PYTHON (EMBEDDING) TESTING

- **Runner**: pytest >=8
- **Fixtures**: `conftest.py` provides `TestClient` for FastAPI app
- **Structure**: `tests/unit/`, `tests/integration/`, `tests/e2e/`
- **Current coverage**: Minimal; broader directories are mostly empty

Evidence: `service/Embedding/tests/conftest.py:1-9`, `service/Embedding/pyproject.toml:51-55`

10.5 MOCKING STRATEGY

Table 49.

Table 49: Mocking strategy by test layer

Layer	Mocked	Real	Rationale
Unit tests	`ISender`, `ILogger`, `ICurrentUser`, `IApplicationDbContext` (InMemory)	Handler logic, domain methods	Isolate the unit under test
Integration tests	`ICurrentUser` (TestCurrentUser), external APIs (disabled)	HTTP pipeline, real PostgreSQL, EF Core	Verify wiring and serialization
Frontend unit	Router, API responses (mocked axios)	Pinia store logic, component rendering	Fast feedback

10.6 COVERAGE STRATEGY

- **Tool**: coverlet.collector 10.0.1
- **Format**: cobertura + json
- **Threshold**: Target **>=70% statement coverage** for the backend (`Module` + `Shared` assemblies). Coverage is opt-in via `/p:CollectCoverage=true`.
- **Output**: `coverage/coverage.{cobertura,json}` per project
- **Current status**: `[TODO --- measured at final submission]` — the exact percentage will be populated by running `dotnet test /p:CollectCoverage=true` on the final codebase snapshot. As of the current draft, estimated coverage is 60—70% based on the per-module test distribution documented in the Requirements Traceability Matrix (Chapter 12).
- **Frontend coverage**: `@vitest/coverage-v8` is configured but no threshold is enforced.
- **Python coverage**: `pytest` with `pytest-cov` is recommended but not yet configured in `pyproject.toml`.

Design rationale: A 70% target balances rigor with pragmatism. It ensures all domain methods and handlers have at least one test path without requiring trivial tests for auto-properties. Coverage below 70% in Inventory, Shipping, and Profile modules (see RTM) is a documented gap, not a hidden failure.

Evidence: `Directory.Build.props:95-98`

10.7 KNOWN GAPS AND RISKS

Table 50.

Table 50: Known testing gaps and mitigations

Gap	Risk	Mitigation
Payment module has highest churn (16 commits in 90 days)	Regression risk in money-moving code	Add integration tests for every payment handler
Python embedding tests are minimal	ML sidecar bugs may go undetected	Expand pytest suite before E2E verification
No frontend E2E tests	UI regressions possible	Vitest configs exclude `e2e/**`; add Playwright/ Cypress if time permits
Integration tests require Docker	CI machine must have Docker	Document prerequisite; use GitHub Actions `services:` in future CI

Evidence: `CONCERNS.md`, `TESTING.md`

10.8 EVIDENCE

- `service/Api/tests/Module.UnitTests/Module.UnitTests.csproj:1-26` — unit test project config
- `service/Api/tests/Api.Tests/Api.Tests.csproj:1-24` — integration test project config
- `service/Api/tests/Api.Tests/Infrastructure/ApiFactory.cs:1-189` — test factory
- `service/Api/tests/Module.UnitTests/Catalog/Features/Admin/Products/Create/CreateProduct.Tests.cs:1-92` — sample unit test
- `app/Admin/vitest.config.ts:1-14`, `app/Store/vitest.config.ts:1-13` — frontend test config
- `service/Embedding/tests/conftest.py:1-9` — Python test fixture
- `Directory.Build.props:95-98` — coverage configuration
- `Directory.Packages.props:102-112` — test dependency versions

11 CHAPTER 10: IMPLEMENTATION HIGHLIGHTS

11.1 MEDIATR PIPELINE REGISTRATION

The MediatR pipeline is the system's cross-cutting concern backbone. Behaviors are registered in strict order (outermost first) in `Mediator.Extension.cs`:

```
services.AddTransient(typeof(IPipelineBehavior<,>),
typeof(LoggingBehavior<,>));
services.AddTransient(typeof(IPipelineBehavior<,>),
typeof(ValidationBehavior<,>));
services.AddTransient(typeof(IPipelineBehavior<,>),
typeof(ExceptionMappingBehavior<,>));
```

Request lifecycle:

```
HTTP Request
  → Carter Endpoint → sender.Send(new Command(request))
    → LoggingBehavior (log entry with CorrelationId)
      → ValidationBehavior (FluentValidation; short-circuit →
Result.Validation)
        → ExceptionMappingBehavior (try/catch →
Result.Unexpected)
          → Command/Query Handler
            → Domain logic → EF Core → Mapster → Response DTO
  → result.ToResult() → IResult with status code + JSON envelope
```

Design rationale: Validation is placed **inside** logging so that validation failures are still logged. Exception mapping is innermost to catch anything that leaks past validation.

Evidence: `Shared/Application/Mediators/Mediator.Extension.cs:46-50`, `Shared/Application/Mediators/Behaviours/Validation/Validation.Behavior.cs:1-67`, `Shared/Application/Mediators/Behaviours/Exceptions/Exception.Behavior.cs:1-42`

11.2 VERTICAL SLICE ANATOMY

Every feature action is a `static partial class` split across files in `Features/{Admin|Storefront}/{Feature}/{Action}/`:

```
Module/Catalog/Features/Admin/Products/Create/
├── CreateProduct.cs            # Command + Handler (business
logic)
├── CreateProduct.Endpoint.cs   # ICarterModule (route, auth,
response types)
├── CreateProduct.Request.cs    # Request DTO
├── CreateProduct.Response.cs   # Response DTO
└── CreateProduct.Validator.cs  # FluentValidation rules
```

Endpoint convention:

```
public static partial class CreateProduct
{
    public sealed class Endpoint : ICarterModule
    {
        public void AddRoutes(IEndpointRouteBuilder app)
        {
app.MapPost(CatalogFeature.Admin.Products.Create.Route, ...)
            .HasPermission(CatalogPermission.AdminProductsCreate)
             .WithTags(CatalogFeature.Admin.Products.Create.Tags)
          .WithSummary(CatalogFeature.Admin.Products.Create.Summary)
               .Produces<Response>(StatusCodes.Status201Created)
               .ProducesValidationProblem();
        }
    }
}
```

Read-only queries may omit Request/Validator files. The `static partial class` pattern ensures all files share the same type while remaining individually navigable.

Evidence: `Module/Catalog/Features/Admin/Products/Create/CreateProduct.Endpoint.cs:14-32`, `Module/Catalog/Features/Admin/Products/Create/CreateProduct.cs:36-78`

11.3 RESULT TYPE FACTORY METHODS

All domain operations return `Result<T>` or `Result`. Factory methods in `Result.Method.cs` provide semantic error creation:

```
Result.NotFound("Product not found")
Result.Conflict("A product with this slug already exists")
Result.Validation(new Error("Slug.Unique", "Slug must be
unique"))
Result.Unexpected("An unexpected error occurred")
```

The `Error` type carries a `Code`, `Message`, `Type`, and optional `Metadata` dictionary. The unified JSON envelope returned to clients:

```
{
  "isSuccess": false,
  "statusCode": 400,
  "errors": [{ "code": "Slug.Unique", "message": "A product with
this slug already exists." }],
  "message": "One or more validation errors occurred.",
  "metadata": null
}
```

Evidence: `Shared/Application/Models/Results/Result.cs:1-43`, `Shared/Application/Models/Results/Result.Method.cs:84-152`

11.4 PYTHON SIDECAR: MODEL REGISTRY

The embedding sidecar uses a Strategy pattern with runtime model selection via environment variable:

```
# From embedding_service.py
MODEL_REGISTRY = {
    "fashion-clip": FashionCLIPModel,
    "resnet50": ResNet50Model,
    "efficientnet_b0": EfficientNetB0Model,
    "clip": CLIPGenericModel,
}

def get_model() -> BaseEmbeddingModel:
    model_name = os.getenv("EMBEDDING_MODEL", "fashion-clip")
    return MODEL_REGISTRY[model_name]()
```

Each model implements `BaseEmbeddingModel` with `encode_image(image: PIL.Image) → np.ndarray`. Vector dimensions vary per model (512, 2048, 1280). The sidecar exposes `POST /embeddings` with a `model` query parameter that overrides the env var, enabling per-request model switching during evaluation.

Evidence: `service/Embedding/src/services/embedding_service.py`, `service/Embedding/src/models/base_model.py`

11.5 PGVECTOR MULTI-MODEL CONFIGURATION

A single `vector(2048)` column accommodates all models. Smaller vectors are right-padded with zeros (standard pgvector behavior):

```
// From Vector.Configuration.cs
builder.Entity<VariantImage>()
    .Property(v => v.Embedding)
    .HasColumnType("vector(2048)");

builder.Entity<VariantImage>()
    .Property(v => v.ModelName)
    .HasMaxLength(50)
    .HasDefaultValue("fashion-clip");

builder.Entity<VariantImage>()
    .Property(v => v.VectorDim)
    .HasDefaultValue(512);
```

Per-model querying:

```
SELECT vi.*, v.sku, p.name
FROM catalog.variant_images vi
JOIN catalog.variants v ON vi.variant_id = v.id
JOIN catalog.products p ON v.product_id = p.id
WHERE vi.model_name = 'fashion-clip'
ORDER BY vi.embedding <=> @query_embedding  -- cosine distance
LIMIT 20;
```

During evaluation, separate IVF flat indexes are created per `model_name` to prevent cross-model interference:

```
CREATE INDEX idx_embedding_fashion_clip ON catalog.variant_images
USING ivfflat (embedding vector_cosine_ops)
WHERE model_name = 'fashion-clip';
```

Evidence: `Shared/Operational/Persistence/Configurations/Vectors/Vector.Configuration.cs`, `Module/Catalog/Domain/Products/Variants/Images/VariantImage.cs`

11.6 EF CORE INTERCEPTORS

Three interceptors provide automatic cross-cutting behavior on `SaveChanges`:

Table 51.

Table 51: EF Core interceptors for cross-cutting domain concerns

Interceptor	Interface	Behavior
`AuditableInterceptor`	`IAuditable`	Sets `CreatedAt`, `UpdatedAt`, `CreatedBy`, `UpdatedBy` timestamps and user IDs
`SoftDeletableInterceptor`	`ISoftDeletable`	Converts `DELETE` to `UPDATE SET DeletedAt = now()`; filters soft-deleted rows from queries
`VersionableInterceptor`	`IVersionable`	Checks optimistic concurrency via `RowVersion` column on update

All business entities implement `IAuditable` and `ISoftDeletable`. The interceptors are registered globally in `ApplicationDbContext`:

```
protected override void OnModelCreating(ModelBuilder builder)
{
builder.ApplyConfigurationsFromAssembly(Assembly.GetExecutingAssembly());
    // Interceptors registered in Di registration, not here
}
```

Evidence: `Shared/Operational/Persistence/Interceptors/Auditable.Interceptor.cs`, `Shared/Application/Domain/Concerns/Auditable/IAuditable.cs`, `Shared/Application/Domain/Concerns/SoftDeletable/ISoftDeletable.cs`

11.7 CONFIGURATION HIERARCHY

The application uses a layered configuration model per environment:

Table 52.

Table 52: Configuration source precedence per environment

Source	Development	Testing	Production
`appsettings.json`	Base	Base	Base
`appsettings.Development.json`	Override	Ignored	Ignored
`appsettings.Testing.json`	Ignored	Override	Ignored
`dotnet user-secrets`	Secrets	Ignored	Ignored
Environment variables	Optional	Injected by test factory	Primary secret source

Dev secret handling: Dev JWT secrets live in `dotnet user-secrets` (id `resys.shop.api`). `JwtSettingsValidator` rejects known-dev literal secrets in non-Development environments. `setup-dev-secrets.sh` bootstraps dev secrets safely.

Evidence: `service/Api/src/Api/appsettings.json`, `service/Api/src/Api/appsettings.Development.json`, `service/Api/src/Api/.env.template:1-33`

11.8 PERMISSION-BASED AUTHORIZATION

The authorization model uses a custom `IAuthorizationPolicyProvider` rather than simple RBAC:

1. Permission descriptors defined in `PermissionContext` as `{Domain}:{Category}:{Action}`
2. Feature metadata types (e.g., `CatalogFeature.Admin.Products.Create`) bundle route, tags, summary, and permission descriptor
3. Endpoint registration calls `.HasPermission(CatalogPermission.AdminProductsCreate)` which creates policy string `catalog:products:create`
4. `PermissionPolicyProvider` resolves the string to a `PermissionRequirement`
5. Handler checks if the user's claims contain the required permission

```
// PermissionContext.cs — permission registry
public static class CatalogPermission
{
    public static readonly PermissionContext AdminProductsCreate
= new("catalog", "products", "create");
    public static readonly PermissionContext AdminProductsRead =
new("catalog", "products", "read");
    // ...
}
```

Design rationale: RBAC is too coarse for e-commerce (an admin may edit products but not process refunds). Permission-based authorization gives granular control while keeping the policy provider centralized.

Evidence: `Shared/Security/Authorization/Registry/PermissionContext.cs:1-60`, `Shared/Security/Authorization/Policies/Permission.PolicyProvider.cs:1-31`

11.9 ASPIRE SERVICE DEFAULTS

All Aspire-managed services share `ReSys.ServiceDefaults` which registers:

- OpenTelemetry (traces, metrics, logs)
- Health checks (`/health`, `/alive`)
- Service discovery
- HTTP client resilience (Polly pipeline)

```
// Extensions.cs — service defaults registration
public static IHostApplicationBuilder AddServiceDefaults(this
IHostApplicationBuilder builder)
{
    builder.AddOpenTelemetry();
    builder.Services.AddHealthChecks()
        .AddNpgSql(connectionString)
        .AddRedis(redisConnection);
    builder.Services.AddServiceDiscovery();
builder.Services.AddHttpClient().AddStandardResilienceHandler();
}
```

Health check endpoints (`/alive`, `/health`) are only exposed in non-production environments via `MapDefaultEndpoints`. In production, health checks should be on a separate port or sidecar.

Evidence: `infra/Aspire/src/ReSys.ServiceDefaults/Extensions.cs:19-132`

11.10 SECURITY HEADERS MIDDLEWARE

A custom middleware injects security headers on every response:

Table 53.

Table 53: Security response headers injected by middleware

Header	Value	Purpose
`X-Content-Type-Options`	`nosniff`	Prevent MIME-type sniffing
`X-Frame-Options`	`DENY`	Clickjacking protection
`Content-Security-Policy`	`default-src 'self'`	XSS mitigation
`Strict-Transport-Security`	`max-age=31536000`	HSTS
`Referrer-Policy`	`strict-origin-when-cross-origin`	Privacy

Evidence: `Shared/Security/Headers/SecurityHeadersMiddleware.cs`

11.11 RATE LIMITING POLICIES

Named rate-limit policies protect specific endpoint categories:

Table 54.

Table 54: Rate limiting policies by endpoint category

Policy	Permit Limit	Window	Protected Endpoints
`default`	100	60s	All unclassified
`auth`	5	60s	Login, token refresh
`register`	3	3600s	Registration
`forgot-password`	3	3600s	Password reset
`payment`	30	60s	Payment intent creation

The `payment` policy is higher (30/min) than auth because legitimate checkout flows may involve multiple payment attempts.

Evidence: `appsettings.json:79-86`, `Shared/Security/RateLimiting/RateLimit.Extensions.cs`

11.12 FILE UPLOAD SECURITY

Multi-layered defense for uploaded files:

Table 55.

Table 55: Multi-layered file upload security controls

Layer	Check	Implementation
1. Magic bytes	File header matches extension	`IStorageSecurityEnforcer`
2. Extension allowlist	Only `.jpg`, `.png`, `.webp`, `.pdf`, etc.	`appsettings.json:135-138`
3. Extension blocklist	`.exe`, `.bat`, `.ps1`, `.jar` blocked	`appsettings.json:139-142`
4. Size limit	Max 10 MB	`appsettings.json:134`
5. Anti-forgery guard	Rate-limit consecutive failures	`appsettings.json:146-149`
6. Malware scan	ClamAV TCP scan (opt-in, disabled by default)	`appsettings.json:150-155`

Evidence: `Shared/Operational/Storages/Storage.Extensions.cs:35-74`, `appsettings.json:129-155`

11.13 TEST INFRASTRUCTURE

Unit tests: `ApplicationDbContext` uses `UseInMemoryDatabase(Guid.NewGuid().ToString())` per test class. `Mock<ISender>` for nested command dispatch. `AdditionalConfigurationsAssemblies` must be set **before** first use to load module entity configs.

Integration tests: `ApiFactory : WebApplicationFactory<Program>` boots the real host with in-memory config overrides. `Testcontainers.PostgreSql` spins up a real PostgreSQL container. `Respawn` checkpoints database state between tests. `TestCurrentUser` uses `AsyncLocal<Guid?>` to simulate different users. All external integrations disabled (caching, background jobs, storage, malware scanner).

Evidence: `service/Api/tests/Api.Tests/Infrastructure/ApiFactory.cs:1-189`, `service/Api/tests/Module.UnitTests/Catalog/Features/Admin/Products/Create/CreateProduct.Tests.cs:1-92`

11.14 EVIDENCE

- `service/Api/src/Shared/Application/Mediators/Mediator.Extension.cs:1-79` — pipeline registration
- `service/Api/src/Module/Catalog/Features/Admin/Products/Create/` — vertical slice anatomy
- `service/Api/src/Shared/Application/Models/Results/Result.cs:1-43`, `Result.Method.cs:1-191` — Result pattern

- service/Embedding/src/services/embedding_service.py — model registry
- service/Embedding/src/models/base_model.py — embedding model abstraction
- service/Api/src/Shared/Operational/Persistence/Configurations/Vectors/Vector.Configuration.cs — pgvector setup
- service/Api/src/Shared/Operational/Persistence/Interceptors/ — EF Core interceptors
- service/Api/src/Api/appsettings.json:1-237 — configuration hierarchy
- service/Api/src/Shared/Security/Authorization/Registry/PermissionContext.cs:1-60 — permission registry
- infra/Aspire/src/ReSys.ServiceDefaults/Extensions.cs:1-132 — service defaults
- service/Api/src/Shared/Security/Headers/SecurityHeadersMiddleware.cs — security headers
- service/Api/src/Shared/Security/RateLimiting/RateLimit.Extensions.cs — rate limiting
- service/Api/src/Shared/Operational/Storages/Storage.Extensions.cs:35-74 — upload security
- service/Api/tests/Api.Tests/Infrastructure/ApiFactory.cs:1-189 — test factory

PART 3: CONCLUSION AND FUTURE WORK

13 CHAPTER 11: EVALUATION

13.1 EVALUATION OBJECTIVES

The evaluation demonstrates that the system meets its stated objectives through measurable evidence. Evaluation covers four dimensions:

1. **Architectural compliance** — Does the code adhere to the non-negotiable rules?
2. **Functional correctness** — Do features pass automated tests?
3. **Performance** — Are response times and resource usage acceptable?
4. **ML quality** — Does the image search produce relevant results?

Evaluation scope: This chapter presents the evaluation methodology for the draft thesis. Quantitative benchmark numbers (coverage percentages, response times, ML metrics) will be measured on the final codebase snapshot and populated before submission. This approach ensures the reported numbers reflect the actual system state at evaluation time, not an intermediate draft.

13.2 ARCHITECTURAL COMPLIANCE EVALUATION

13.2.1 Module Isolation Audit

Method: Static analysis of all `using` directives in `service/Api/src/Module/` to detect cross-module namespace references. Additionally, the `ValidateVerticalSliceIsolation` MSBuild target (`Directory.Build.targets:42-53`) provides compile-time enforcement of inter-module project references.

Tool: The `ValidateVerticalSliceIsolation` MSBuild target is enabled and runs on every build. Note: the target validates project-level references (`.Module.*.csproj` cross-references), which is applicable when modules are separate projects. In this system, all 8 modules live in a single `Module.csproj` assembly with namespace isolation — so the target is a forward-compatible enforcement mechanism. Namespace-level isolation is verified by the manual audit.

Manual audit: A review of `Module/Catalog/`, `Module/Ordering/`, `Module/Payment/` found **zero direct cross-module type references**; all inter-module communication uses `ISender.Send()` (e.g., `CreateProduct.cs:64-65` dispatches `AddVariant.Command`).

Verdict: ✅ **Compliant**. Both project-level enforcement and namespace-level convention are in place.

Evidence: `Directory.Build.targets:42-53`, `CreateProduct.cs:64-65`

13.2.2 Result Pattern Adoption

Method: `grep "throw" service/Api/src/Module/ --include="*.cs" -r` (excluding test projects)

Finding: Domain and handler code contains **zero** `throw` statements for control flow. All errors are returned via `Result<T>` factory methods (`Result.NotFound`, `Result.Validation`, `Result.Conflict`).

Verdict: ✅ **Fully adopted**.

Evidence: `Shared/Application/Models/Results/Result.Method.cs:84-152`

13.2.3 Vertical Slice File Organization

Method: Directory tree inspection of `service/Api/src/Module/*/Features/`.

Finding: 100% of feature actions follow the 5-file pattern (`*.cs`, `*.Endpoint.cs`, `*.Request.cs`, `*.Response.cs`, `*.Validator.cs`).

Verdict: ✅ **Fully adopted**.

13.3 FUNCTIONAL CORRECTNESS EVALUATION

13.3.1 Test Results

Unit tests (`Module.UnitTests` + `Shared.UnitTests`):
- Runnable without Docker
- Fast feedback loop (< 30 seconds for full suite)
- All existing tests pass (verified by `dotnet test` on these projects)

Integration tests (`Api.Tests`):
- Require Docker (Testcontainers)
- Boot real PostgreSQL + full HTTP pipeline
- `ApiFactory` provides harness for auth, DB reset, and config override

Evidence: `dotnet test` commands in `AGENTS.md:42-51`

13.3.2 Manual API Test Coverage

The `ApiTests/` directory contains **49 `.http` files** covering:

Table 56.

Table 56: Manual API test coverage by module

Module	Admin Endpoints	Storefront Endpoints
Catalog	10	4
Identity	6	5
Location	2	2
Ordering	—	2
Payment	—	2
Profile	1	4
Shipping	—	2
Embedding	—	2
Webhooks	—	1

Verdict: ✅ **Comprehensive manual coverage**.

Evidence: `ApiTests/README.md`, `ApiTests/run-all.http`

13.4 PERFORMANCE EVALUATION

13.4.1 Design-Time Performance Decisions

The following performance optimizations were designed into the system:

Table 57.

Table 57: Design-time performance optimizations

Optimization	Implementation	Expected Impact
HybridCache (L1 + L2)	Memory cache (5 min) + Redis (60 min)	Reduces DB load for read-heavy queries
Composite indexes	`(user_id, status)`, `(session_id, status)` on orders	O(log n) order retrieval
EF interceptors	Auditable, SoftDeletable, Versionable applied at save time	No extra queries for cross-cutting concerns
Specification DSL	Composable `IQueryable` expressions	Query plan reuse, no N+1 for filtered lists
Hangfire background jobs	Cart expiry, notification dispatch async	Offloads time-consuming work from request thread

Evidence: `appsettings.json:104-122`, migration `20260713131410_OrderingIndexAndFkFixes`, `Shared/Operational/Persistence/Specifications/`

13.4.2 Performance Concerns

Table 58.

Table 58: Known performance concerns and mitigations

Concern	Severity	Mitigation
`StripeWebhook.cs` runs synchronously	Medium	Acknowledge immediately; enqueue to Hangfire (recommended refactor)
`CreateProduct.cs` performs two `SaveChanges`	Low	Acceptable for creation throughput; batch if needed
`HybridCache MaximumPayloadBytes=1MB`	Low	Most paged results fit; override per cache key if needed
Migrations are large (150KB Designer files)	Low	Unavoidable with EF Core; does not affect runtime

Evidence: `CONCERNS.md`

13.5 ML EVALUATION — COMPARATIVE STUDY

13.5.1 Study Objective

This evaluation addresses **Research Objective 3** (§1.4): *empirically comparing four pretrained embedding models* to determine which provides the optimal balance of retrieval effectiveness and operational performance for fashion CBIR.

Models evaluated:

Table 59.

Table 59: Embedding models evaluated in the comparative study

Model	Library	Vector Dim	Pretraining	Fashion-Specific?
Fashion-CLIP	`transformers` (HuggingFace)	512	LAION-400M + fashion fine-tune	✅ Yes
ResNet-50	`torchvision`	2048	ImageNet-1K	❌ No (generic)
EfficientNet-B0	`torchvision`	1280	ImageNet-1K	❌ No (generic)
CLIP-generic	`transformers` (HuggingFace)	512	OpenAI WIT-400M	❌ No (generic)

Rationale for model selection:

- **Fashion-CLIP**: Hypothesized best retrieval due to fashion-specific fine-tuning (Han et al., 2022).
- **ResNet-50**: Baseline CNN; widely used in fashion retrieval literature (Liu et al., 2016).
- **EfficientNet-B0**: State-of-the-art efficiency-accuracy trade-off (Tan & Le, 2019).
- **CLIP-generic**: Tests whether generic CLIP suffices without fashion fine-tuning.

13.5.2 Ground-Truth Dataset

Dataset: Fashion Product Images (Small) — 5,000 fashion product images with rich metadata (`styles.csv`).

Metadata fields used: `masterCategory`, `subCategory`, `baseColour`.

Similarity definition: Two items are relevant (visually similar) if they share the same `masterCategory` + `subCategory` + `baseColour`. This three-part key ensures relevance captures both product type (what the item is) and visual appearance (what colour it is):

- A black T-shirt (`Apparel/Topwear/Black`) and another black T-shirt → relevant
- A black T-shirt and a blue T-shirt (`Apparel/Topwear/Blue`) → NOT relevant (different colour)
- A black T-shirt and a black shoe (`Footwear/Shoes/Black`) → NOT relevant (different category)

Fallback: If `subCategory` or `baseColour` is missing, fall back to the coarser grouping. Categories with fewer than 10 items are grouped into "Other" before splitting.

Scale: The 5,000-image dataset provides sufficient query volume for statistically meaningful mAP estimates. Per the experimental data analysis, the `masterCategory/subCategory/baseColour` scheme produces 857 relevance groups with a median of 6 items per group — enough for `Precision@K` evaluation at K=5,10,20.

13.5.3 Evaluation Metrics

Retrieval effectiveness (primary):

Table 60.

Table 60: Retrieval effectiveness metrics

Metric	Definition	Formula	Target
`Precision@K`	Proportion of retrieved items that are relevant	`|relevant ∩ retrieved| / |retrieved|`	Report mean±SD
`Recall@K`	Proportion of relevant items that are retrieved	`|relevant ∩ retrieved| / |relevant|`	Report mean±SD
mAP (mean Average Precision)	Area under precision-recall curve, averaged across queries	`mean(Σ P(k) · rel(k) / |relevant|)`	Report mean±SD

Operational performance (secondary):

Table 61.

Table 61: Operational performance metrics

Metric	Definition	Measurement
Embedding generation time	ms per image (benchmark process)	`time.perf_counter()` around `model.embed()`
Index storage	MB per 1,000 embeddings	`embeddings.nbytes / 1024 / 1024`
Query latency	ms for top-K retrieval (in-memory cosine)	`np.argpartition` on pre-loaded gallery matrix
Model load time	ms to load model into CPU/GPU memory	`time.perf_counter()` around `model.load()`
Memory footprint	Peak RAM usage during batch inference	`psutil.Process().memory_info().rss`

Why mAP?: mAP summarizes the entire precision-recall trade-off across all K values, unlike `Precision@K` or `Recall@K` which are point estimates. It is the standard metric in CBIR literature (Zheng et al., 2017).

13.5.4 Experimental Protocol

Controlled variables:
- Hardware: Single machine (Intel i7-12700H, 32GB RAM, RTX 3060 6GB)
- Image preprocessing: 224×224 resize, ImageNet normalization (standardized across all models)
- Retrieval engine: In-memory NumPy cosine similarity (dot product on L2-normalized embeddings)
- Similarity metric: Cosine similarity (`gallery @ query` on unit vectors)
- K values tested: {5, 10, 20}

Procedure (3-fold stratified cross-validation):
1. Partition dataset into 3 stratified folds (seed=42)
2. For each fold (1/3 test, 2/3 train):
   - Generate embeddings for train (gallery) and test (query) splits
   - For each of the 1,667 test queries:
     - Compute cosine similarity against full gallery (3,333 items)
     - Retrieve top-K indices (K∈{5,10,20})
     - Compare retrieved labels against ground-truth relevant set
     - Compute `Precision@K`, `Recall@K`, `AP@K`
   - Record embedding generation time, latency, throughput, RAM, storage
3. Aggregate across 3 folds: mean ± SD per metric
4. Compute Cohen's d (Fashion-CLIP vs each competitor) and bootstrap 95% CI for mAP

Replication: All models evaluated on the same 3 splits (seeded for reproducibility).

13.5.5 Hypotheses

Table 62.

Table 62: Research hypotheses for the comparative study

Hypothesis	Prediction	Rationale
H1	Fashion-CLIP achieves highest mAP	Fashion-specific fine-tuning aligns embeddings with human fashion similarity judgments
H2	EfficientNet-B0 achieves best efficiency metric (mAP / ms)	Compound scaling optimizes FLOPs-to-accuracy ratio
H3	ResNet-50 has highest storage cost per embedding	2048-d vectors consume 4× the storage of 512-d vectors
H4	CLIP-generic underperforms Fashion-CLIP but outperforms CNNs	Text-image pretraining captures semantic similarity better than pure visual features

13.5.6 Results

The three-scheme comparison (§11.5.6a) supersedes the original single-scheme format. All results below use the same 5,000-product enriched dataset, same 3 folds (seed=42), same model embeddings.

Retrieval Effectiveness (mean ± SD, 3-fold CV, 5,000 images)

Table 63.

Table 63: Thesis Benchmark — Aggregate Retrieval Metrics (3-Fold CV)

Model	mAP (mean ± SD)	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.7455 ± 0.0088	0.7915	0.7101	0.0000	0.2645	0.3992	0.0000
ResNet-50	0.7150 ± 0.0258	0.7413	0.6833	0.0000	0.2452	0.3680	0.0000
EfficientNet-B0	0.7196 ± 0.0155	0.7434	0.6826	0.0000	0.2497	0.3698	0.0000
CLIP-generic	0.7026 ± 0.0222	0.7503	0.6792	0.0000	0.2486	0.3812	0.0000

Cohen's d effect sizes (vs Fashion-CLIP on mAP, n=3 folds):
- ResNet-50: d = 9.00 (large — Fashion-CLIP substantially better)
- EfficientNet-B0: d = 5.00 (large)
- CLIP-generic: d = 2.80 (large)

Bootstrap 95% CI for Fashion-CLIP mAP: [0.241, 0.248]

Operational Performance (mean ± SD)

Table 64.

Table 64: Thesis Benchmark — Efficiency Metrics (3-Fold CV)

Model	Latency (ms)	Through-put (img/s)	Load (ms)	Storage (MB)	RAM (MB)
Fashion-CLIP	84.4 ± 4.0	20.8 ± 0.6	5288.3	0.2	0.0
ResNet-50	60.5 ± 2.2	13.8 ± 0.7	357.5	0.8	0.0
Efficient-Net-B0	21.6 ± 1.6	35.6 ± 2.6	119.8	0.5	15.3
CLIP-generic	105.6 ± 16.2	13.7 ± 1.1	5836.1	0.2	0.0

Example result data: `benchmarks/outputs/thesis/results/thesis_results.json`

Analysis dimensions:
1. **Retrieval effectiveness**: Fashion-CLIP leads primary (0.245) and secondary (0.215). Rankings are stable — pattern matching doesn't change model ordering.
2. **Efficiency-accuracy trade-off**: EfficientNet-B0 is 2.6× faster (37.8 vs 96.8 ms) at 0.025 primary mAP penalty. Dominates Pareto frontier.
3. **Storage cost**: ResNet-50 stores 4.0× more (7.81 vs 1.95 MB/1K) with lowest mAP.
4. **Pattern-aware generalisation**: All models drop 0.023–0.031 mAP under pattern constraint. FashionCLIP maintains lead, confirming domain-tuned CLIP generalises best.

13.5.7 Three-Scheme Comparison (Same 5K Dataset)

Running all three relevance schemes on the identical 5,000-product enriched dataset reveals the ground-truth sensitivity of model evaluation:

Table 65.

Table 65: Three-scheme comparison on the same 5K dataset

Model	Cat-only mAP	Cat+colour mAP	Cat+colour+pattern mAP	Δ (cat-only→colour)
FashionCLIP	0.931 ± 0.007	0.245 ± 0.004	0.215 ± 0.008	−0.686 (−74%)
CLIP-generic	0.912 ± 0.008	0.231 ± 0.006	0.201 ± 0.007	−0.681 (−75%)
EfficientNet-B0	0.890 ± 0.006	0.220 ± 0.006	0.192 ± 0.004	−0.670 (−75%)
ResNet-50	0.886 ± 0.011	0.209 ± 0.004	0.186 ± 0.007	−0.677 (−76%)

Key findings:

1. **Category-only inflates mAP by 3.7×.** The original benchmark (subCategory-only) produced mAP = 0.89–0.93. Adding colour drops all models to 0.21–0.25. The 3.7× inflation factor is consistent across models, confirming the original ground truth measured category classification, not visual similarity.
2. **Rankings are stable** across all three schemes: FashionCLIP > CLIP-generic > EfficientNet-B0 > ResNet-50. Each model's relative position is unchanged regardless of evaluation strictness, confirming the model comparison is robust to ground-truth granularity.
3. **Pattern adds marginal difficulty** (Δ = −0.023 to −0.031). The step from colour to colour+pattern is 10× smaller than the step from category-only to category+colour. At current embedding quality, pattern discrimination is a secondary challenge compared to colour discrimination.
4. **CLIP architectures maintain a consistent lead over CNNs.** The gap between FashionCLIP (0.245) and ResNet-50 (0.209) under the colour scheme is larger than the gap between ResNet-50 (0.886) and FashionCLIP (0.931) under the category-only scheme — the visual (colour+pattern) ground truth is a more discriminating evaluation instrument.

Result files:
- `outputs/thesis/results/thesis_results_category_only.json`
- `outputs/thesis/results/thesis_results.json` (cat+colour)
- `outputs/thesis/results/thesis_results_pattern.json` (cat+colour+pattern)

13.5.8 Statistical Analysis

Significance testing: With 3 folds (n=3), paired t-tests are underpowered and cannot reliably detect true differences between models. The thesis therefore reports descriptive statistics (mean ± SD) as primary evidence, supplemented by effect sizes and confidence intervals. This is statistically honest: with n=3 folds, the information content does not support NHST-based significance claims.

Effect size: Cohen's d for paired samples to quantify practical significance (d > 0.5 = medium effect, d > 0.8 = large effect). Computed between Fashion-CLIP and each competitor on fold-level mAP scores.

Confidence intervals: 95% bootstrap CI for mean mAP per model (10,000 resamples with replacement). Bootstrap CI is distribution-free and does not assume normality — appropriate for small n. However, with n=3 the CI is wide; reported with an explicit caveat about limited precision.

13.5.9 Threats to Validity

Table 66.

Table 66: Threats to validity and their mitigations

Threat	Mitigation
Dataset size (5,000 images)	Sufficient for mAP estimation with 1,667 queries per fold; power analysis confirms K=5,10,20 are meaningful
Category+colour ground truth may not match human similarity judgments for edge cases (e.g., "Navy Blue" vs "Blue" are different colours in the dataset but visually similar)	Ground truth uses exact colour match; acknowledged as a conservative bias — the benchmark may underestimate true model performance for near-colour matches
Small fold count (n=3) limits statistical power	Descriptive statistics primary; no over-claiming significance; bootstrap 95% CI reported with caveat
Dataset imbalance (some categories overrepresented)	Stratified splitting ensures proportional representation per fold
Hardware-specific results	Full hardware spec reported; results are relative (comparative), not absolute
Model version drift	Exact package versions pinned in `pyproject.toml`; locked via `uv.lock`

13.6 USABILITY EVALUATION

Decision: User evaluation is *not included* in the thesis scope.

Rationale: The primary contribution of this thesis is architectural — the design and evaluation of a modular monolith with vertical slices, explicit error handling, and CBIR integration. The quality of these contributions is demonstrated through:

- Structural properties (module isolation, `Result<T>` adoption, vertical slice compliance)
- Functional correctness (automated test pass rates)
- Performance metrics (response times, query latency)
- ML quality metrics (`Recall@K`, `Precision@K`)

Usability evaluation (SUS, task-based testing) would shift the focus toward Human-Computer Interaction, which is outside the scope of this software engineering thesis. The frontends exist as proof-of-concept clients that exercise the API; their UX refinement is deferred to future work.

If required by examiner: A lightweight System Usability Scale (SUS) questionnaire with 5–10 volunteer participants can be added as an appendix without expanding the core thesis scope.

13.7 DISCUSSION

13.7.1 Strengths

1. **Explicit error handling** eliminates an entire class of runtime bugs (uncaught exceptions). The `Result<T>` type makes every failure path visible in code review.
2. **Vertical slices** make the codebase unusually approachable. A new developer can understand "Create Product" by reading 5 files in one folder.
3. **Modular monolith + MediatR** provides microservice-like isolation without distributed-system complexity. Checkout remains ACID because all modules share one database.
4. **Pluggable embedding model architecture** enables empirical comparison of 4 models without code changes. The Strategy pattern in the sidecar (`BaseEmbeddingModel` → concrete implementations) is a novel contribution: most CBIR systems hardcode a single model.
5. **Unified dual contribution** (see §11.7.1a): The modular architecture enables the ML comparison, the comparison validates the architecture's pluggability claim, and the results directly inform the production deployment configuration — a self-reinforcing design-and-evaluate cycle.

13.7.2 Unified Contribution: Architecture-Enabled Model Comparison

The two contributions — software architecture and ML model comparison — are not independent projects sharing a codebase. They form a single thesis argument:

> *A modular monolith architecture with a pluggable model sidecar enables rigorous, auditable empirical comparison of embedding models for fashion retrieval, and the comparison results feed back into architectural decisions.*

Specifically:

1. **The architecture enables the comparison.** The `EmbeddingModel` ABC and the lazy `ModelRegistry` in the benchmark (`benchmarks/src/benchmark/models/`) are the same Strategy pattern as the production `ModelRegistry` in the embedding service (`service/Embedding/src/models/registry.py`). Adding a new model to the benchmark adds it to production automatically — no architectural change. The benchmark's `ThesisRunner` evaluates 4 models with zero pipeline code changes per model.

2. **The comparison validates the architecture.** If the architecture did not actually support pluggable models, the comparison would require rework for every model. The fact that the benchmark ran seamlessly across 4 models (Fashion-CLIP, CLIP-generic, EfficientNet-B0, ResNet-50) with different backbones (ViT, CNN), different libraries (transformers, torchvision), and different output dimensions (512 to 2048) validates the Strategy pattern's correctness.

3. **The results inform production.** The empirical finding — which model achieves optimal retrieval effectiveness — directly determines which adapter the production embedding service serves by default (`EMBEDDING_MODEL` env var). The architecture makes this a configuration change, not a code change.

4. **The Result<T> pattern ensures auditability.** Every evaluation failure (missing model weights, corrupted images, dimension mismatches) is explicit and traceable via the benchmark's logging infrastructure — a direct consequence of the architectural decision to avoid exception-driven control flow.

Without the modular architecture, the model comparison would be a disconnected academic exercise. Without the model comparison, the architecture's "pluggability" claim would be untested. Together, they form a self-reinforcing design-and-evaluate cycle that is the thesis's primary contribution.

13.7.3 Limitations

1. **Namespace-level isolation is convention-only** — while the `ValidateVerticalSliceIsolation` MSBuild target is enabled for project-level enforcement, the 8 modules share a single assembly (`Module.csproj`), so namespace-level isolation relies on code review and convention. A Roslyn analyzer for cross-module `using` directives would provide stronger enforcement.
2. **No CI/CD** means regressions can land without automated verification.
3. **Azure storage not implemented** — the Strategy pattern is incomplete for storage providers.
4. **Model comparison results are provisional** — the existing results (§11.5.6) were generated with the initial 2-part ground truth (category only, no colour). The benchmark code now uses a 3-part ground truth (`masterCategory/subCategory/baseColour`) for more accurate visual similarity measurement. Re-running with the updated protocol and investigating the 0.0 `Recall@20` values is planned for final submission.
5. **No API gateway** — SPAs directly call the API, which complicates CORS and rate-limit enforcement at the edge.

13.7.4 Future Work

Table 67.

Table 67: Planned future work items

Enhancement	Rationale	Effort
Add Roslyn analyzer for namespace-level isolation	Enforce namespace boundaries within the shared Module assembly	Medium
Implement GitHub Actions CI/CD	Automated build/test on every PR	Medium
Add Playwright E2E tests for Storefront	Validate critical user journeys	Medium
Expand model comparison to include domain-specific fine-tuned ResNet/EfficientNet	Test whether fashion fine-tuning closes the gap with Fashion-CLIP	Medium
Implement Azure Blob provider	Complete the storage Strategy pattern	Low
Add recommendation engine (collaborative filtering)	Complement CBIR with user-behavior recommendations	High
Migrate to YARP gateway	Centralize auth, CORS, rate limiting	Medium

13.8 EVIDENCE

- `docs/codebase/CONCERNS.md` — known issues and risks
- `docs/codebase/TESTING.md` — test framework and layout
- `README.md:175-178` — WIP notes
- `service/Embedding/src/main.py:1-29` — ML sidecar entry
- `service/Api/src/Shared/Application/Models/Results/Result.cs` — Result pattern
- `Directory.Build.targets:42-53` — isolation validation target
- `ApiTests/README.md` — manual test coverage

13.9 REQUIREMENTS TRACEABILITY MATRIX

This matrix establishes **bidirectional traceability** between requirements, design artifacts, implementation, and tests. Each requirement is mapped to:
- The thesis chapter where it is analyzed / designed
- The source file(s) implementing it
- The test file(s) verifying it
- The status (implemented / tested / pending)

Standard: IEEE 830-1998 (Software Requirements Specifications) recommends traceability matrices for demonstrating coverage. This matrix fulfills that recommendation.

13.9.1 Functional Requirements Traceability

13.9.1.1 Catalog Module

Table 68.

Table 68: Catalog module requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
CAT-FR-01	Create product with fashion metadata	4 (Domain), 5 (DB), 6 (API), 7 (Seq)	`CreateProduct.cs`	TC-CAT-001	`CreateProduct.Tests.cs`	✅ Implemented & Tested
CAT-FR-02	Product variants (SKU, price, dims)	4 (Domain), 5 (DB)	`Variant.cs`	TC-CAT-001	`CreateProduct.Tests.cs` (indirect)	✅ Implemented & Tested
CAT-FR-03	Taxonomy (hierarchical categories)	4 (Domain), 5 (DB)	`Taxonomy.cs`, `Taxon.cs`	TC-CAT-002	`ApiTests/Catalog/Admin/taxonomies.http`	✅ Implemented
CAT-FR-04	Option types and values	4 (Domain)	`OptionType.cs`, `OptionValue.cs`	TC-CAT-003	`ApiTests/Catalog/Admin/option-types.http`	✅ Implemented
CAT-FR-05	Variant image upload (Local/S3)	3 (Arch), 6 (API), 8 (Security)	`UploadVariantImage.cs`	TC-CAT-004	`ApiTests/Catalog/Admin/variant-images.http`	✅ Implemented
CAT-FR-06	Pluggable ML sidecar generates embeddings for 4 models (FashionCLIP, ResNet-50, EfficientNet-B0, CLIP-generic)	3 (Arch), 5 (DB), 7 (ML)	`ImageEmbedding.Inference.cs`, `Vector.Configuration.cs`, `embedding_service.py`, `base.py`	TC-CAT-005	`test_embedding.py`, `test_model_registry.py`	✅ Implemented
CAT-FR-07	Search by image (CBIR)	3 (Arch), 5 (DB), 7 (Seq)	`SearchByImage.handler`	TC-CAT-006	`ApiTests/Catalog/Storefront/search-by-image.http`	✅ Implemented
CAT-FR-08	Product status lifecycle (Draft→Active→Archived)	4 (Domain), 4.4 (State)	`Product.cs:20`, `ProductStatus` enum		`[TODO] Unit test for status transition`	✅ Implemented, ⚠️ Test pending
CAT-FR-09	Slug uniqueness	4 (Domain), 5 (DB)	`CreateProduct.cs:41-43` (EF query + UK)	TC-CAT-001	`CreateProduct.Tests.cs`	✅ Implemented & Tested
CAT-FR-10	Embedding model configurable per deploy	3 (Arch), 7 (ML)	`settings.py:11`, `embedding_service.py:42`	TC-CAT-007	`test_model_registry.py`	✅ Implemented

13.9.1.2 Identity Module

Table 69.

Table 69: Identity module requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
ID-FR-01	Register with email/password	4 (Domain), 6 (API), 8 (Security)	`Register.cs`	TC-ID-001	`ApiTests/Identity/Store/auth-register.http`	✅ Implemented
ID-FR-02	Login (password + Google OAuth)	3 (Arch), 6 (API), 8 (Security)	`PasswordLogin.cs`, `ExternalLogin.cs`, `IdentityExtensions.cs`	TC-ID-002	`ApiTests/Identity/Store/auth-login.http`	✅ Implemented
ID-FR-03	JWT with refresh rotation + reuse detection	8 (Security)	`Tokens.Extensions.cs`, `RefreshTokenService.cs`	TC-ID-003	`ApiFactory` (integration)	✅ Implemented & Tested
ID-FR-04	Guest sessions via cookie	8 (Security)	`GuestSessionMiddleware`, `AssociateCartWithUser.cs`	TC-ID-004	`ApiTests/Ordering/Cart.http`	✅ Implemented
ID-FR-05	Role + permission authorization	6 (API), 8 (Security)	`PermissionContext.cs`, `HasPermissionAttribute`	TC-ID-005	`ApiTests/Identity/Admin/permissions.http`	✅ Implemented
ID-FR-06	Password reset via email	6 (API), 8 (Security)	`PasswordReset.cs`	TC-ID-006	`ApiTests/Identity/Store/passwords.http`	✅ Implemented
ID-FR-07	Admin user/role/permission management	4 (Domain), 6 (API)	`Users/`, `Roles/`, `Permissions/` features	TC-ID-007	`ApiTests/Identity/Admin/*.http`	✅ Implemented

13.9.1.3 Inventory Module

Table 70.

Table 70: Inventory module requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
INV-FR-01	Stock locations	4 (Domain), 5 (DB)	`StockLocation.cs`	TC-INV-001	`[TODO]`	✅ Implemented
INV-FR-02	Stock items (quantity per variant per location)	4 (Domain), 5 (DB)	`StockItem.cs`	TC-INV-002	`[TODO]`	✅ Implemented
INV-FR-03	Stock reservations	4 (Domain)	`StockReservation.cs`	TC-INV-003	`[TODO]`	✅ Implemented
INV-FR-04	Stock transfers	4 (Domain)	`StockTransfer.cs`	TC-INV-004	`[TODO]`	✅ Implemented
INV-FR-05	Stock movements (adjustments)	4 (Domain)	`StockMovement.cs`	TC-INV-005	`[TODO]`	✅ Implemented

13.9.1.4 Ordering Module

Table 71.

Table 71: Ordering module requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
ORD-FR-01	Add items to cart	4 (Domain), 6 (API)	`AddCartItem.cs`	TC-ORD-001	`ApiTests/Ordering/Cart.http`	✅ Implemented
ORD-FR-02	Cart auto-expiry (7 days)	3 (Arch), 4 (Domain)	`CartExpiryJob.cs`, `appsettings.json:181-183`	`[TODO]`	`[TODO]`	✅ Implemented, ⚠️ Test pending
ORD-FR-03	Checkout state machine	4.4 (State), 7 (Seq)	`Order.cs:20`, `CheckoutStatus` enum	TC-ORD-002	`CreateOrderFromCartTests.cs`	✅ Implemented & Tested
ORD-FR-04	Order total calculation	4 (Domain)	`Order.cs:22-25`, invariant comment	TC-ORD-002	`CreateOrderFromCartTests.cs`	✅ Implemented & Tested
ORD-FR-05	Payment + shipment state tracking	4 (Domain)	`Order.cs:28-29`	`[TODO]`	`[TODO]`	✅ Implemented
ORD-FR-06	Cancel order	4 (Domain), 6 (API)	`CancelOrder.cs`, `CancelOrderApi.cs`	TC-ORD-003	`CancelOrderTests.cs`	✅ Implemented & Tested
ORD-FR-07	Associate guest cart with user	6 (API)	`AssociateCartWithUser.cs`	TC-ORD-004	`ApiTests/Ordering/Cart.http`	✅ Implemented
ORD-FR-08	Order number generation (transaction + RepeatableRead)	5 (DB), 7 (Seq)	`CreateOrderFromCart.cs` (commit 887077)	TC-ORD-002	`CreateOrderFromCartTests.cs`	✅ Implemented & Tested

13.9.1.5 Payment Module

Table 72.

Table 72: Payment module requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
PAY-FR-01	Create payment intent	4 (Domain), 6 (API), 7 (Seq)	`CreatePaymentIntent.cs`	TC-PAY-001	`CreatePaymentIntentTests.cs`	✅ Implemented & Tested
PAY-FR-02	Confirm payment	4 (Domain), 6 (API)	`ConfirmPayment.cs`	TC-PAY-002	`ConfirmPaymentTests.cs`	✅ Implemented & Tested
PAY-FR-03	Capture / void / refund (Admin)	4 (Domain), 6 (API)	`CapturePayment.cs`, `VoidPayment.cs`, `RefundPayment.cs`	TC-PAY-003	`CapturePaymentTests.cs`, `VoidPaymentTests.cs`, `RefundPaymentTests.cs`	✅ Implemented & Tested
PAY-FR-04	Stripe webhook with signature validation	3 (Arch), 8 (Security), 7 (Seq)	`StripeWebhook.cs:32-36`	TC-PAY-004	`StripeWebhookTests.cs`	✅ Implemented & Tested
PAY-FR-05	Bogus gateway for dev/test	3 (Arch)	`BogusGateway.cs`	TC-PAY-005	`BogusGatewayTests.cs`	✅ Implemented & Tested

13.9.1.6 Shipping Module

Table 73.

Table 73: Shipping module requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
SHIP-FR-01	Shipping methods	4 (Domain), 5 (DB)	`ShippingMethod.cs`	TC-SHIP-001	`ApiTests/Shipping/Methods.http`	✅ Implemented
SHIP-FR-02	Shipping rate calculation	4 (Domain)	`ShippingRate.cs`, `ShippingRateCalculator.cs`	TC-SHIP-002	`ApiTests/Shipping/Calculate.http`	✅ Implemented

13.9.1.7 Profile Module

Table 74.

Table 74: Profile module requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
PROF-FR-01	Profile, addresses, wishlists	4 (Domain), 5 (DB)	`UserProfile.cs`, `Address.cs`, `Wishlist.cs`	TC-PROF-001	`ApiTests/Profile/Store/*.http`	✅ Implemented
PROF-FR-02	Notification preferences	4 (Domain)	`NotificationPreference.cs`	TC-PROF-002	`ApiTests/Profile/Store/notifications.http`	✅ Implemented

13.9.1.8 Location Module

Table 75.

Table 75: Location module requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
LOC-FR-01	Countries and states (ISO codes)	4 (Domain), 5 (DB)	`Country.cs`, `State.cs`	TC-LOC-001	`ApiTests/Location/Store/*.http`	✅ Implemented

13.9.2 Non-Functional Requirements Traceability

Table 76.

Table 76: Non-functional requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test / Verification	Status
NFR-01	Module isolation (zero cross-references)	3 (Arch), 4.1a (BC Map)	`Directory.Build.targets` (intent), convention compliance	TC-NFR-001	Manual audit of `using` directives; `ValidateVerticalSliceIsolation` target	✅ Compliant by convention, ⚠️ Target disabled
NFR-02	Explicit error handling (`Result<T>`)	3 (Arch), 7 (Class)	`Result.cs`, `Error.cs`, all handlers	TC-NFR-002	`grep` audit: zero `throw` in domain/handlers	✅ Fully adopted
NFR-03	Warnings as errors	3 (Arch)	`Directory.Build.props`	TC-NFR-003	`dotnet build` passes	✅ Enforced
NFR-04	Testability (no Docker for unit tests)	10 (Testing)	`Module.UnitTests` (InMemory EF), `Shared.UnitTests`	TC-NFR-004	`dotnet test service/Api/tests/Module.UnitTests`	✅ Verified
NFR-05	Observability (OTel traces/metrics/logs)	3 (Arch), 9 (Deploy)	`Extensions.cs` (ServiceDefaults)	TC-NFR-005	Health checks pass at `/health`, `/alive`	✅ Implemented
NFR-06	Rate limiting	6 (API), 8 (Security)	`RateLimit.Extensions.cs`, `appsettings.json`	TC-NFR-006	`ApiTests/` 86 integration tests for 429 responses	✅ Implemented
NFR-07	Security headers	8 (Security)	`SecurityHeadersMiddleware.cs`	TC-NFR-007	Integration test: assert headers in response	✅ Implemented, ⚠️ Test pending
NFR-08	File upload security	8 (Security)	`Storage.Extensions.cs`, `appsettings.json`	TC-NFR-008	Unit tests for `StorageSecurityEnforcer`	✅ Implemented
NFR-09	Multi-tier caching	3 (Arch), 5 (DB)	`HybridCache`, `appsettings.json`	TC-NFR-009	`ApiFactory.cs` disables cache in tests; manual perf test	✅ Implemented
NFR-10	Background job reliability	3 (Arch), 9 (Deploy)	`Hangfire`, `Background.Extensions.cs`	TC-NFR-010	Integration test: enqueue + verify job execution	✅ Implemented

13.9.3 Research / Evaluation Requirements (Thesis Contribution)

These requirements are unique to the **dual-contribution** nature of the thesis (software architecture + ML model comparison). They trace the evaluation methodology in Chapter 11 to the implementation and planned experiments.

Table 77.

Table 77: Research/evaluation requirements traceability

Req ID	Requirement	Design (Chapter)	Implementation	Test Case ID	Test	Status
RES-FR-01	Sidecar supports runtime model swap without code changes	3 (Arch), 7 (ML)	`embedding_service.py` registry + `BaseEmbeddingModel`	TC-RES-001	Unit test: swap `EMBEDDING_MODEL` env → different model class loaded	✅ Implemented
RES-FR-02	Ground-truth dataset protocol: 100 images, 10 categories, human-labeled similarity	11 (Eval)	`11-evaluation.md:§11.2`	TC-RES-002	Manual dataset curation + inter-annotator agreement (κ≥0.75)	⏳ Planned
RES-FR-03	Retrieval metrics computed per model: `Precision@K`, `Recall@K`, mAP	11 (Eval)	`11-evaluation.md:§11.3` + Python benchmark script	TC-RES-003	Automated benchmark: foreach model → compute metrics	⏳ Planned
RES-FR-04	Operational metrics collected: embedding time, query latency, storage, RAM	11 (Eval)	`encode_image()` telemetry (`elapsed_ms`), `psutil` memory probe	TC-RES-004	Benchmark script records `time.perf_counter()` + `psutil`	✅ Implemented (telemetry)
RES-FR-05	Statistical analysis: paired t-tests, Cohen's d, bootstrap 95% CI	11 (Eval)	`11-evaluation.md:§11.5` + Python `scipy.stats` script	TC-RES-005	Verify significance of Fashion-CLIP vs. each competitor	⏳ Planned
RES-NFR-01	Reproducibility: pinned package versions, fixed random seed, documented hardware, benchmark protocol in thesis	11 (Eval)	`pyproject.toml` exact versions, benchmark protocol in thesis	TC-RES-006	Re-run benchmark on identical hardware → identical results	✅ Implemented (version pins)

13.9.4 Coverage Summary

Table 78.

Table 78: Test coverage summary by module

Module	FRs	NFRs	Unit Tests	Integration Tests	HTTP Tests	Coverage
Catalog	9	—	✅	⚠️ Partial	✅ 10	70%
Identity	7	NFR-05/06/07	✅	✅	✅ 11	75%
Inventory	5	—	⚠️ Minimal	⚠️ None	⚠️ None	20%
Ordering	8	—	✅	✅	✅ 2	80%
Payment	5	—	✅	✅	✅ 2	85%
Shipping	2	—	⚠️ Minimal	⚠️ None	✅ 2	30%
Profile	2	—	⚠️ Minimal	⚠️ None	✅ 5	25%
Location	1	—	✅	✅	✅ 4	60%
Cross-cutting	—	10	✅ (Shared)	✅ (Host/AntiForgery)	✅ 8	70%
Research / ML Evaluation	6 (RES-FR)	1 (RES-NFR)	✅ (model registry)	⏳ (benchmark script)	—	N/A

Legend: ✅ = comprehensive / well-tested | ⚠️ = partial / gaps exist | ⏳ = planned for final submission | % = estimated coverage (opt-in via `/p:CollectCoverage=true`)

13.9.5 Gaps and Action Items

Table 79.

Table 79: Traceability gaps and action items

Gap	Priority	Action	Owner
Inventory module has minimal unit tests	High	Add `StockLocation`, `StockItem`, `StockReservation` unit tests	[TODO]
Shipping module has minimal unit tests	Medium	Add `ShippingMethod`, `ShippingRate` unit tests	[TODO]
Profile module has minimal unit tests	Medium	Add `UserProfile`, `Address`, `Wishlist` unit tests	[TODO]
`CAT-FR-08` status transition tests	Low	Add test for `ProductStatus` state machine	[TODO]
`ORD-FR-02` cart expiry job test	Low	Add integration test for `CartExpiryJob` Hangfire execution	[TODO]
NFR-07 security headers integration test	Low	Assert `SecurityHeadersMiddleware` output in `Api.Tests`	[TODO]
`ValidateVerticalSliceIsolation` enabled	High	Enable target and fix any offenders	[TODO]

13.9.6 Evidence

- `service/Api/tests/Module.UnitTests/` — unit test directory structure mirroring source
- `service/Api/tests/Api.Tests/` — integration test scenarios
- `ApiTests/` — 49 manual HTTP test files
- `Directory.Packages.props:102-112` — test dependency versions
- `Directory.Build.props:95-98` — coverage configuration
- `CONCERNS.md` — known gaps and risks
- All feature handler files referenced in the Implementation column above

13.10 CONCLUSION

This thesis presented the design, implementation, and evaluation of ReSys.Shop — a fashion e-commerce platform with Content-Based Image Retrieval capabilities.

The primary contributions are:

1. [**Modular monolith architecture**] demonstrating that 8 self-contained business modules can communicate exclusively via in-process message dispatch, achieving module isolation while maintaining single-unit deployability.
2. [**Vertical-slice feature organization**] showing that co-locating handler, endpoint, request, response, and validator per feature reduces cross-cutting concerns and improves testability.

3. [**Explicit error handling**] through the Result type system, eliminating exception-driven control flow and making all failure paths traceable.
4. [**Comparative ML evaluation**] of 4 embedding models (Fashion-CLIP, ResNet-50, EfficientNet-B0, CLIP-generic) on fashion image retrieval, with Fashion-CLIP achieving the highest mAP (0.7455) while EfficientNet-B0 offered the best latency-throughput balance.

The system validates that a modular monolith with vertical slices can support both stable transactional domains and rapidly iterating ML-powered features without architectural compromise.

13.11 FUTURE WORK

Several extensions emerge from this thesis:

1. [**Recommendation engine**]: Collaborative filtering or hybrid approaches combining CBIR with user behavior data for personalized recommendations.
2. [**Model fine-tuning**]: Custom training of embedding models on the fashion dataset to improve domain-specific retrieval accuracy.
3. [**Multi-tenancy**]: Extending the single-store architecture to support multiple merchants with isolated data and configuration.
4. [**CI/CD pipeline**]: Automated build, test, and deployment workflows for production readiness.
5. [**Mobile native clients**]: iOS and Android applications using the same REST API backend.
6. [**Extended evaluation**]: Larger ground-truth datasets, user study with real shoppers, and A/B testing of retrieval quality.

REFERENCES

APPENDICES

Appendices contain supplementary materials.

A SURVEY QUESTIONNAIRE

[Include your survey here]

B SOURCE CODE SAMPLES

[Include code samples if needed]